

17. Classes

A class is a data structure that may contain data members (constants and fields), function members (methods, properties, events, indexers, operators, instance constructors, destructors, and static constructors), and nested types. Class types support inheritance, a mechanism whereby a **derived class** can extend and specialize a **base class**.

17.1 Class declarations

A *class-declaration* is a *type-declaration* (§16.5) that declares a new class.

class-declaration:

*attributes*_{opt} *class-modifiers*_{opt} **class** *identifier* *class-base*_{opt} *class-body* **;**_{opt}

A *class-declaration* consists of an optional set of *attributes* (§24), followed by an optional set of *class-modifiers* (§17.1.1), followed by the keyword **class** and an *identifier* that names the class, followed by an optional *class-base* specification (§17.1.2), followed by a *class-body* (§17.1.3), optionally followed by a semicolon.

17.1.1 Class modifiers

A *class-declaration* may optionally include a sequence of class modifiers:

class-modifiers:

class-modifier

class-modifiers *class-modifier*

class-modifier:

new

public

protected

internal

private

abstract

sealed

It is a compile-time error for the same modifier to appear multiple times in a class declaration.

The **new** modifier is permitted on nested classes. It specifies that the class hides an inherited member by the same name, as described in §10.2.2. It is a compile-time error for the **new** modifier to appear on a class declaration that is not a nested class declaration.

The **public**, **protected**, **internal**, and **private** modifiers control the accessibility of the class. Depending on the context in which the class declaration occurs, some of these modifiers may not be permitted (§10.5.1).

The **abstract** and **sealed** modifiers are discussed in the following sections.

17.1.1.1 Abstract classes

The **abstract** modifier is used to indicate that a class is incomplete and that it is intended to be used only as a base class. An **abstract class** differs from a **non-abstract class** in the following ways:

- An abstract class cannot be instantiated directly, and it is a compile-time error to use the `new` operator on an abstract class. While it is possible to have variables and values whose compile-time types are abstract, such variables and values will necessarily either be `null` or contain references to instances of non-abstract classes derived from the abstract types.
- An abstract class is permitted (but not required) to contain abstract members.
- An abstract class cannot be sealed.

When a non-abstract class is derived from an abstract class, the non-abstract class must include actual implementations of all inherited abstract members, thereby overriding those abstract members. [*Example:* In the example

```

abstract class A
{
    public abstract void F();
}

abstract class B: A
{
    public void G() {}
}

class C: B
{
    public override void F() {
        // actual implementation of F
    }
}

```

the abstract class `A` introduces an abstract method `F`. Class `B` introduces an additional method `G`, but since it doesn't provide an implementation of `F`, `B` must also be declared abstract. Class `C` overrides `F` and provides an actual implementation. Since there are no abstract members in `C`, `C` is permitted (but not required) to be non-abstract. *end example*]

17.1.1.2 Sealed classes

The `sealed` modifier is used to prevent derivation from a class. A compile-time error occurs if a sealed class is specified as the base class of another class.

A sealed class cannot also be an abstract class.

[*Note:* The `sealed` modifier is primarily used to prevent unintended derivation, but it also enables certain run-time optimizations. In particular, because a sealed class is known to never have any derived classes, it is possible to transform virtual function member invocations on sealed class instances into non-virtual invocations. *end note*]

17.1.2 Class base specification

A class declaration may include a *class-base* specification, which defines the direct base class of the class and the interfaces (§20) implemented by the class.

```

class-base:
    : class-type
    : interface-type-list
    : class-type , interface-type-list

interface-type-list:
    interface-type
    interface-type-list , interface-type

```

17.1.2.1 Base classes

When a *class-type* is included in the *class-base*, it specifies the direct base class of the class being declared. If a class declaration has no *class-base*, or if the *class-base* lists only interface types, the direct base class is assumed to be `object`. A class inherits members from its direct base class, as described in §17.2.1.

[*Example:* In the example

```
class A {}
class B: A {}
```

class A is said to be the direct base class of B, and B is said to be derived from A. Since A does not explicitly specify a direct base class, its direct base class is implicitly `object`. *end example*]

The direct base class of a class type must be at least as accessible as the class type itself (§10.5.4). For example, it is a compile-time error for a `public` class to derive from a `private` or `internal` class.

The direct base class of a class type must not be any of the following types: `System.Array`, `System.Delegate`, `System.Enum`, or `System.ValueType`.

The base classes of a class are the direct base class and its base classes. In other words, the set of base classes is the transitive closure of the direct base class relationship. [*Note:* Referring to the example above, the base classes of B are A and `object`. *end note*]

Except for class `object`, every class has exactly one direct base class. The `object` class has no direct base class and is the ultimate base class of all other classes.

When a class B derives from a class A, it is a compile-time error for A to depend on B. A class *directly depends on* its direct base class (if any) and *directly depends on* the class within which it is immediately nested (if any). Given this definition, the complete set of classes upon which a class depends is the transitive closure of the *directly depends on* relationship.

[*Example:* The example

```
class A: B {}
class B: C {}
class C: A {}
```

is in error because the classes circularly depend on themselves. Likewise, the example

```
class A: B.C {}
class B: A
{
    public class C {}
}
```

results in a compile-time error because A depends on B.C (its direct base class), which depends on B (its immediately enclosing class), which circularly depends on A. *end example*]

Note that a class does not depend on the classes that are nested within it. [*Example:* In the example

```
class A
{
    class B: A {}
}
```

B depends on A (because A is both its direct base class and its immediately enclosing class), but A does not depend on B (since B is neither a base class nor an enclosing class of A). Thus, the example is valid. *end example*]

It is not possible to derive from a `sealed` class. [*Example:* In the example

```
sealed class A {}
class B: A {}           // Error, cannot derive from a sealed class
```

class B results in a compile-time error because it attempts to derive from the `sealed` class A. *end example*]

17.1.2.2 Interface implementations

A *class-base* specification may include a list of interface types, in which case the class is said to implement the given interface types. Interface implementations are discussed further in §20.4.

17.1.3 Class body

The *class-body* of a class defines the members of that class.

```
class-body:
    { class-member-declarationsopt }
```

17.2 Class members

The members of a class consist of the members introduced by its *class-member-declarations* and the members inherited from the direct base class.

```
class-member-declarations:
    class-member-declaration
    class-member-declarations class-member-declaration

class-member-declaration:
    constant-declaration
    field-declaration
    method-declaration
    property-declaration
    event-declaration
    indexer-declaration
    operator-declaration
    constructor-declaration
    destructor-declaration
    static-constructor-declaration
    type-declaration
```

The members of a class are divided into the following categories:

- Constants, which represent constant values associated with that class (§17.3).
- Fields, which are the variables of that class (§17.4).
- Methods, which implement the computations and actions that can be performed by that class (§17.5).
- Properties, which define named characteristics and the actions associated with reading and writing those characteristics (§17.6).
- Events, which define notifications that can be generated by that class (§17.7).
- Indexers, which permit instances of that class to be indexed in the same way as arrays (§17.8).
- Operators, which define the expression operators that can be applied to instances of that class (§17.9).
- Instance constructors, which implement the actions required to initialize instances of that class (§17.10).
- Destructors, which implement the actions to be performed before instances of that class are permanently discarded (§17.12).
- Static constructors, which implement the actions required to initialize that class itself (§17.11).
- Types, which represent the types that are local to that class (§16.5).

Members that can contain executable code are collectively known as the *function members* of the class. The function members of a class are the methods, properties, events, indexers, operators, instance constructors, destructors, and static constructors of that class.

A *class-declaration* creates a new declaration space (§10.3), and the *class-member-declarations* immediately contained by the *class-declaration* introduce new members into this declaration space. The following rules apply to *class-member-declarations*:

- Instance constructors, destructors, and static constructors must have the same name as the immediately enclosing class. All other members must have names that differ from the name of the immediately enclosing class.
- The name of a constant, field, property, event, or type must differ from the names of all other members declared in the same class.
- The name of a method must differ from the names of all other non-methods declared in the same class. In addition, the signature (§10.6) of a method must differ from the signatures of all other methods declared in the same class.
- The signature of an instance constructor must differ from the signatures of all other instance constructors declared in the same class.
- The signature of an indexer must differ from the signatures of all other indexers declared in the same class.
- The signature of an operator must differ from the signatures of all other operators declared in the same class.

The inherited members of a class (§17.2.1) are not part of the declaration space of a class. [*Note:* Thus, a derived class is allowed to declare a member with the same name or signature as an inherited member (which in effect hides the inherited member). *end note*]

17.2.1 Inheritance

A class ***inherits*** the members of its direct base class. Inheritance means that a class implicitly contains all members of its direct base class, except for the instance constructors, destructors, and static constructors of the base class. Some important aspects of inheritance are:

- Inheritance is transitive. If C is derived from B, and B is derived from A, then C inherits the members declared in B as well as the members declared in A.
- A derived class *extends* its direct base class. A derived class can add new members to those it inherits, but it cannot remove the definition of an inherited member.
- Instance constructors, destructors, and static constructors are not inherited, but all other members are, regardless of their declared accessibility (§10.5). However, depending on their declared accessibility, inherited members might not be accessible in a derived class.
- A derived class can ***hide*** (§10.7.1.2) inherited members by declaring new members with the same name or signature. Note however that hiding an inherited member does not remove that member—it merely makes that member inaccessible in the derived class.
- An instance of a class contains a set of all instance fields declared in the class and its base classes, and an implicit conversion (§13.1.4) exists from a derived class type to any of its base class types. Thus, a reference to an instance of some derived class can be treated as a reference to an instance of any of its base classes.
- A class can declare virtual methods, properties, and indexers, and derived classes can override the implementation of these function members. This enables classes to exhibit polymorphic behavior wherein the actions performed by a function member invocation varies depending on the run-time type of the instance through which that function member is invoked.

17.2.2 The new modifier

A *class-member-declaration* is permitted to declare a member with the same name or signature as an inherited member. When this occurs, the derived class member is said to ***hide*** the base class member. Hiding

an inherited member is not considered an error, but it does cause the compiler to issue a warning. To suppress the warning, the declaration of the derived class member can include a `new` modifier to indicate that the derived member is intended to hide the base member. This topic is discussed further in §10.7.1.2.

If a `new` modifier is included in a declaration that doesn't hide an inherited member, a warning to that effect is issued. This warning is suppressed by removing the `new` modifier.

17.2.3 Access modifiers

A *class-member-declaration* can have any one of the five possible kinds of declared accessibility (§10.5.1): `public`, `protected internal`, `protected`, `internal`, or `private`. Except for the `protected internal` combination, it is a compile-time error to specify more than one access modifier. When a *class-member-declaration* does not include any access modifiers, `private` is assumed.

17.2.4 Constituent types

Types that are used in the declaration of a member are called the *constituent types* of that member. Possible constituent types are the type of a constant, field, property, event, or indexer, the return type of a method or operator, and the parameter types of a method, indexer, operator, or instance constructor. The constituent types of a member must be at least as accessible as that member itself (§10.5.4).

17.2.5 Static and instance members

Members of a class are either *static members* or *instance members*. [Note: Generally speaking, it is useful to think of static members as belonging to classes and instance members as belonging to objects (instances of classes). *end note*]

When a field, method, property, event, operator, or constructor declaration includes a `static` modifier, it declares a static member. In addition, a constant or type declaration implicitly declares a static member. Static members have the following characteristics:

- When a static member is referenced in a *member-access* (§14.5.4) of the form `E.M`, `E` must denote a type that has a member `M`. It is a compile-time error for `E` to denote an instance.
- A static field identifies exactly one storage location. No matter how many instances of a class are created, there is only ever one copy of a static field.
- A static function member (method, property, event, operator, or constructor) does not operate on a specific instance, and it is a compile-time error to refer to `this` in such a function member.

When a field, method, property, event, indexer, constructor, or destructor declaration does not include a `static` modifier, it declares an instance member. (An instance member is sometimes called a non-static member.) Instance members have the following characteristics:

- When an instance member is referenced in a *member-access* (§14.5.4) of the form `E.M`, `E` must denote an instance of a type that has a member `M`. It is a compile-time error for `E` to denote a type.
- Every instance of a class contains a separate set of all instance fields of the class.
- An instance function member (method, property, indexer, instance constructor, or destructor) operates on a given instance of the class, and this instance can be accessed as `this` (§14.5.7).

[Example: The following example illustrates the rules for accessing static and instance members:

```
class Test
{
    int x;
    static int y;

    void F() {
        x = 1;           // Ok, same as this.x = 1
        y = 1;           // Ok, same as Test.y = 1
    }
}
```

```

1      static void G() {
2          x = 1;           // Error, cannot access this.x
3          y = 1;           // Ok, same as Test.y = 1
4      }
5
6      static void Main() {
7          Test t = new Test();
8          t.x = 1;         // Ok
9          t.y = 1;         // Error, cannot access static member through
instance
10         Test.x = 1;       // Error, cannot access instance member through type
11         Test.y = 1;       // Ok
12     }
13 }

```

14 The `F` method shows that in an instance function member, a *simple-name* (§14.5.2) can be used to access
15 both instance members and static members. The `G` method shows that in a static function member, it is a
16 compile-time error to access an instance member through a *simple-name*. The `Main` method shows that in a
17 *member-access* (§14.5.4), instance members must be accessed through instances, and static members must
18 be accessed through types. *end example*]

19 17.2.6 Nested types

20 A type declared within a class or struct is called a **nested type**. A type that is declared within a compilation
21 unit or namespace is called a **non-nested type**. [*Example:* In the following example:

```

22     using System;
23     class A
24     {
25         class B
26         {
27             static void F() {
28                 Console.WriteLine("A.B.F");
29             }
30         }
31     }

```

32 class `B` is a nested type because it is declared within class `A`, and class `A` is a non-nested type because it is
33 declared within a compilation unit. *end example*]

34 17.2.6.1 Fully qualified name

35 The fully qualified name (§10.8.1) for a nested type is `S.N` where `S` is the fully qualified name of the type in
36 which type `N` is declared.

37 17.2.6.2 Declared accessibility

38 Non-nested types can have public or internal declared accessibility and they internal declared accessibility
39 by default. Nested types can have these forms of declared accessibility too, plus one or more additional
40 forms of declared accessibility, depending on whether the containing type is a class or struct:

- 41 • A nested type that is declared in a class can have any of five forms of declared accessibility (public,
42 protected internal, protected, internal, or private) and, like other class members, defaults to private declared
43 accessibility.
- 44 • A nested type that is declared in a struct can have any of three forms of declared accessibility (public,
45 internal, or private) and, like other struct members, defaults to private declared accessibility.

46 [*Example:* The example

```

1      public class List
2      {
3          // Private data structure
4          private class Node
5          {
6              public object Data;
7              public Node Next;
8              public Node(object data, Node next) {
9                  this.Data = data;
10                 this.Next = next;
11             }
12         }
13
14         private Node first = null;
15         private Node last = null;
16
17         // Public interface
18         public void AddToFront(object o) {...}
19         public void AddToBack(object o) {...}
20         public object RemoveFromFront() {...}
21         public object AddToFront() {...}
22         public int Count { get {...} }
23     }

```

declares a private nested class Node. *end example*]

17.2.6.3 Hiding

A nested type may hide (§10.7.1.1) a base member. The **new** modifier is permitted on nested type declarations so that hiding can be expressed explicitly. [*Example*: The example

```

26      using System;
27      class Base
28      {
29          public static void M() {
30              Console.WriteLine("Base.M");
31          }
32      }
33
34      class Derived: Base
35      {
36          new public class M
37          {
38              public static void F() {
39                  Console.WriteLine("Derived.M.F");
40              }
41          }
42
43          class Test
44          {
45              static void Main() {
46                  Derived.M.F();
47              }
48          }

```

shows a nested class M that hides the method M defined in Base. *end example*]

17.2.6.4 this access

A nested type and its containing type do not have a special relationship with regard to *this-access* (§14.5.7). Specifically, **this** within a nested type cannot be used to refer to instance members of the containing type. In cases where a nested type needs access to the instance members of its containing type, access can be provided by providing the **this** for the instance of the containing type as a constructor argument for the nested type. [*Example*: The following example


```

1      using System;
2      class C
3      {
4          int i = 123;
5          public void F() {
6              Nested n = new Nested(this);
7              n.G();
8          }
9          public class Nested {
10             C this_c;
11             public Nested(C c) {
12                 this_c = c;
13             }
14             public void G() {
15                 Console.WriteLine(this_c.i);
16             }
17         }
18     }
19     class Test {
20         static void Main() {
21             C c = new C();
22             c.F();
23         }
24     }

```

25 shows this technique. An instance of `C` creates an instance of `Nested`, and passes its own `this` to `Nested`'s
 26 constructor in order to provide subsequent access to `C`'s instance members. *end example*

27 17.2.6.5 Access to private and protected members of the containing type

28 A nested type has access to all of the members that are accessible to its containing type, including members
 29 of the containing type that have private and protected declared accessibility. [*Example*: The example

```

30     using System;
31     class C
32     {
33         private static void F() {
34             Console.WriteLine("C.F");
35         }
36         public class Nested
37         {
38             public static void G() {
39                 F();
40             }
41         }
42     }
43     class Test
44     {
45         static void Main() {
46             C.Nested.G();
47         }
48     }

```

49 shows a class `C` that contains a nested class `Nested`. Within `Nested`, the method `G` calls the static method `F`
 50 defined in `C`, and `F` has private declared accessibility. *end example*

51 A nested type also may access protected members defined in a base type of its containing type. [*Example*: In
 52 the example

```

53     using System;
54     class Base
55     {
56         protected void F() {
57             Console.WriteLine("Base.F");
58         }
59     }

```

```

1      class Derived: Base
2      {
3          public class Nested
4          {
5              public void G() {
6                  Derived d = new Derived();
7                  d.F();      // ok
8              }
9          }
10     }
11     class Test
12     {
13         static void Main() {
14             Derived.Nested n = new Derived.Nested();
15             n.G();
16         }
17     }

```

18 the nested class `Derived.Nested` accesses the protected method `F` defined in `Derived`'s base class, `Base`,
 19 by calling through an instance of `Derived`. *end example*]

20 17.2.7 Reserved member names

21 To facilitate the underlying C# runtime implementation, for each source member declaration that is a
 22 property, event, or indexer, the implementation must reserve two method signatures based on the kind of the
 23 member declaration, its name, and its type (§17.2.7.1, §17.2.7.2, §17.2.7.3). It is a compile-time error for a
 24 program to declare a member whose signature matches one of these reserved signatures, even if the
 25 underlying runtime implementation does not make use of these reservations.

26 The reserved names do not introduce declarations, thus they do not participate in member lookup. However,
 27 a declaration's associated reserved method signatures do participate in inheritance (§17.2.1), and can be
 28 hidden with the `new` modifier (§17.2.2).

29 [Note: The reservation of these names serves three purposes:

- 30 1. To allow the underlying implementation to use an ordinary identifier as a method name for get or set
 31 access to the C# language feature.
- 32 2. To allow other languages to interoperate using an ordinary identifier as a method name for get or set
 33 access to the C# language feature.
- 34 3. To help ensure that the source accepted by one conforming compiler is accepted by another, by
 35 making the specifics of reserved member names consistent across all C# implementations.

36 *end note*]

37 The declaration of a destructor (§17.12) also causes a signature to be reserved (§17.2.7.4).

38 17.2.7.1 Member Names Reserved for Properties

39 For a property `P` (§17.6) of type `T`, the following signatures are reserved:

```

40     T get_P();
41     void set_P(T value);

```

42 Both signatures are reserved, even if the property is read-only or write-only.

43 [Example: In the example

```

44     using System;
45     class A {
46         public int P {
47             get { return 123; }
48         }
49     }

```

```

1      class B: A {
2          new public int get_P() {
3              return 456;
4          }
5          new public void set_P(int value) {
6          }
7      }

8      class Test
9      {
10         static void Main() {
11             B b = new B();
12             A a = b;
13             Console.WriteLine(a.P);
14             Console.WriteLine(b.P);
15             Console.WriteLine(b.get_P());
16         }
17     }

```

18 a class A defines a read-only property P, thus reserving signatures for `get_P` and `set_P` methods. A class B
19 derives from A and hides both of these reserved signatures. The example produces the output:

```

20         123
21         123
22         456

```

23 *end example]*

24 17.2.7.2 Member Names Reserved for Events

25 For an event E (§17.7) of delegate type T, the following signatures are reserved:

```

26         void add_E(T handler);
27         void remove_E(T handler);

```

28 17.2.7.3 Member Names Reserved for Indexers

29 For an indexer (§17.8) of type T with parameter-list L, the following signatures are reserved:

```

30         T get_Item(L);
31         void set_Item(L, T value);

```

32 Both signatures are reserved, even if the indexer is read-only or write-only.

33 17.2.7.4 Member Names Reserved for Destructors

34 For a class containing a destructor (§17.12), the following signature is reserved:

```

35         void Finalize();

```

36 17.3 Constants

37 A **constant** is a class member that represents a constant value: a value that can be computed at compile-time.

38 A *constant-declaration* introduces one or more constants of a given type.

```

39     constant-declaration:
40         attributesopt constant-modifiersopt const type constant-declarators ;

41     constant-modifiers:
42         constant-modifier
43         constant-modifiers constant-modifier

44     constant-modifier:
45         new
46         public
47         protected
48         internal
49         private

```

```

1      constant-declarators:
2          constant-declarator
3          constant-declarators , constant-declarator
4
5      constant-declarator:
6          identifier = constant-expression

```

A *constant-declaration* may include a set of *attributes* (§24), a *new* modifier (§17.2.2), and a valid combination of the four access modifiers (§17.2.3). The attributes and modifiers apply to all of the members declared by the *constant-declaration*. Even though constants are considered static members, a *constant-declaration* neither requires nor allows a *static* modifier. It is an error for the same modifier to appear multiple times in a constant declaration.

The *type* of a *constant-declaration* specifies the type of the members introduced by the declaration. The type is followed by a list of *constant-declarators*, each of which introduces a new member. A *constant-declarator* consists of an *identifier* that names the member, followed by an “=” token, followed by a *constant-expression* (§14.15) that gives the value of the member.

The *type* specified in a constant declaration must be *sbyte*, *byte*, *short*, *ushort*, *int*, *uint*, *long*, *ulong*, *char*, *float*, *double*, *decimal*, *bool*, *string*, an *enum-type*, or a *reference-type*. Each *constant-expression* must yield a value of the target type or of a type that can be converted to the target type by an implicit conversion (§13.1).

The *type* of a constant must be at least as accessible as the constant itself (§10.5.4).

The value of a constant is obtained in an expression using a *simple-name* (§14.5.2) or a *member-access* (§14.5.4).

A constant can itself participate in a *constant-expression*. Thus, a constant may be used in any construct that requires a *constant-expression*. [Note: Examples of such constructs include *case* labels, *goto case* statements, *enum* member declarations, attributes, and other constant declarations. *end note*]

[Note: As described in §14.15, a *constant-expression* is an expression that can be fully evaluated at compile-time. Since the only way to create a non-null value of a *reference-type* other than *string* is to apply the *new* operator, and since the *new* operator is not permitted in a *constant-expression*, the only possible value for constants of *reference-types* other than *string* is *null*. *end note*]

When a symbolic name for a constant value is desired, but when the type of that value is not permitted in a constant declaration, or when the value cannot be computed at compile-time by a *constant-expression*, a *readonly* field (§17.4.2) may be used instead. [Note: The versioning semantics of *const* and *readonly* differ (§17.4.2.2). *end-note*]

A constant declaration that declares multiple constants is equivalent to multiple declarations of single constants with the same attributes, modifiers, and type. [Example: For example

```

35      class A
36      {
37          public const double x = 1.0, y = 2.0, z = 3.0;
38      }

```

is equivalent to

```

40      class A
41      {
42          public const double x = 1.0;
43          public const double y = 2.0;
44          public const double z = 3.0;
45      }

```

end example]

Constants are permitted to depend on other constants within the same program as long as the dependencies are not of a circular nature. The compiler automatically arranges to evaluate the constant declarations in the appropriate order. [Example: In the example

```

1      class A
2      {
3          public const int X = B.Z + 1;
4          public const int Y = 10;
5      }
6
7      class B
8      {
9          public const int Z = A.Y + 1;
10     }

```

the compiler first evaluates `A.Y`, then evaluates `B.Z`, and finally evaluates `A.X`, producing the values 10, 11, and 12. *end example*] Constant declarations may depend on constants from other programs, but such dependencies are only possible in one direction. [*Example:* Referring to the example above, if `A` and `B` were declared in separate programs, it would be possible for `A.X` to depend on `B.Z`, but `B.Z` could then not simultaneously depend on `A.Y`. *end example*]

17.4 Fields

A **field** is a member that represents a variable associated with an object or class. A *field-declaration* introduces one or more fields of a given type.

```

18     field-declaration:
19         attributesopt field-modifiersopt type variable-declarators ;
20
21     field-modifiers:
22         field-modifier
23         field-modifiers field-modifier
24
25     field-modifier:
26         new
27         public
28         protected
29         internal
30         private
31         static
32         readonly
33         volatile
34
35     variable-declarators:
36         variable-declarator
37         variable-declarators , variable-declarator
38
39     variable-declarator:
40         identifier
41         identifier = variable-initializer
42
43     variable-initializer:
44         expression
45         array-initializer

```

A *field-declaration* may include a set of *attributes* (§24), a `new` modifier (§17.2.2), a valid combination of the four access modifiers (§17.2.3), and a `static` modifier (§17.4.1). In addition, a *field-declaration* may include a `readonly` modifier (§17.4.2) or a `volatile` modifier (§17.4.3), but not both. The attributes and modifiers apply to all of the members declared by the *field-declaration*. It is an error for the same modifier to appear multiple times in a *field declaration*. It is an error for the same modifier to appear multiple times in a field declaration.

The *type* of a *field-declaration* specifies the type of the members introduced by the declaration. The type is followed by a list of *variable-declarators*, each of which introduces a new member. A *variable-declarator* consists of an *identifier* that names that member, optionally followed by an “=” token and a *variable-initializer* (§17.4.5) that gives the initial value of that member.

The *type* of a field must be at least as accessible as the field itself (§10.5.4).

The value of a field is obtained in an expression using a *simple-name* (§14.5.2) or a *member-access* (§14.5.4). The value of a non-readonly field is modified using an *assignment* (§14.13). The value of a non-readonly field can be both obtained and modified using postfix increment and decrement operators (§14.5.9) and prefix increment and decrement operators (§14.6.5).

A field declaration that declares multiple fields is equivalent to multiple declarations of single fields with the same attributes, modifiers, and type. [Example: For example

```
class A
{
    public static int x = 1, y, z = 100;
}
```

is equivalent to

```
class A
{
    public static int x = 1;
    public static int y;
    public static int z = 100;
}
```

end example]

17.4.1 Static and instance fields

When a field declaration includes a `static` modifier, the fields introduced by the declaration are **static fields**. When no `static` modifier is present, the fields introduced by the declaration are **instance fields**. Static fields and instance fields are two of the several kinds of variables (§12) supported by C#, and at times they are referred to as **static variables** and **instance variables**, respectively.

A static field is not part of a specific instance; instead, it identifies exactly one storage location. No matter how many instances of a class are created, there is only ever one copy of a static field for the associated application domain.

An instance field belongs to an instance. Specifically, every instance of a class contains a separate set of all the instance fields of that class.

When a field is referenced in a *member-access* (§14.5.4) of the form `E.M`, if `M` is a static field, `E` must denote a type that has a field `M`, and if `M` is an instance field, `E` must denote an instance of a type that has a field `M`.

The differences between static and instance members are discussed further in §17.2.5.

17.4.2 Readonly fields

When a *field-declaration* includes a `readonly` modifier, the fields introduced by the declaration are **readonly fields**. Direct assignments to readonly fields can only occur as part of that declaration or in an instance constructor or static constructor in the same class. (A readonly field can be assigned to multiple times in these contexts.) Specifically, direct assignments to a `readonly` field are permitted only in the following contexts:

- In the *variable-declarator* that introduces the field (by including a *variable-initializer* in the declaration).
- For an instance field, in the instance constructors of the class that contains the field declaration; for a static field, in the static constructor of the class that contains the field declaration. These are also the only contexts in which it is valid to pass a `readonly` field as an `out` or `ref` parameter.

Attempting to assign to a `readonly` field or pass it as an `out` or `ref` parameter in any other context is a compile-time error.

17.4.2.1 Using static readonly fields for constants

A **static readonly** field is useful when a symbolic name for a constant value is desired, but when the type of the value is not permitted in a **const** declaration, or when the value cannot be computed at compile-time. [Example: In the example

```

5      public class Color
6      {
7          public static readonly Color Black = new Color(0, 0, 0);
8          public static readonly Color White = new Color(255, 255, 255);
9          public static readonly Color Red = new Color(255, 0, 0);
10         public static readonly Color Green = new Color(0, 255, 0);
11         public static readonly Color Blue = new Color(0, 0, 255);
12
13         private byte red, green, blue;
14         public Color(byte r, byte g, byte b) {
15             red = r;
16             green = g;
17             blue = b;
18         }
19     }

```

the **Black**, **White**, **Red**, **Green**, and **Blue** members cannot be declared as **const** members because their values cannot be computed at compile-time. However, declaring them **static readonly** instead has much the same effect. *end example*]

17.4.2.2 Versioning of constants and static readonly fields

Constants and readonly fields have different binary versioning semantics. When an expression references a constant, the value of the constant is obtained at compile-time, but when an expression references a readonly field, the value of the field is not obtained until run-time. [Example: Consider an application that consists of two separate programs:

```

27     using System;
28     namespace Program1
29     {
30         public class Utils
31         {
32             public static readonly int X = 1;
33         }
34     }
35
36     namespace Program2
37     {
38         class Test
39         {
40             static void Main() {
41                 Console.WriteLine(Program1.Utils.X);
42             }
43         }
44     }

```

The **Program1** and **Program2** namespaces denote two programs that are compiled separately. Because **Program1.Utils.X** is declared as a static readonly field, the value output by the **Console.WriteLine** statement is not known at compile-time, but rather is obtained at run-time. Thus, if the value of **X** is changed and **Program1** is recompiled, the **Console.WriteLine** statement will output the new value even if **Program2** isn't recompiled. However, had **X** been a constant, the value of **X** would have been obtained at the time **Program2** was compiled, and would remain unaffected by changes in **Program1** until **Program2** is recompiled. *end example*]

17.4.3 Volatile fields

When a *field-declaration* includes a **volatile** modifier, the fields introduced by that declaration are **volatile fields**. For non-volatile fields, optimization techniques that reorder instructions can lead to unexpected and unpredictable results in multi-threaded programs that access fields without synchronization

such as that provided by the *lock-statement* (§15.12). These optimizations can be performed by the compiler, by the runtime system, or by hardware. For volatile fields, such reordering optimizations are restricted:

- A read of a volatile field is called a **volatile read**. A volatile read has “acquire semantics”; that is, it is guaranteed to occur prior to any references to memory that occur after it in the instruction sequence.
- A write of a volatile field is called a **volatile write**. A volatile write has “release semantics”; that is, it is guaranteed to happen after any memory references prior to the write instruction in the instruction sequence.

These restrictions ensure that all threads will observe volatile writes performed by any other thread in the order in which they were performed. A conforming implementation is not required to provide a single total ordering of volatile writes as seen from all threads of execution. The type of a volatile field must be one of the following:

- A *reference-type*.
- The type `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `char`, `float`, or `bool`.
- An *enum-type* having an enum base type of `byte`, `sbyte`, `short`, `ushort`, `int`, or `uint`.

[*Example*: The example

```
using System;
using System.Threading;
class Test
{
    public static int result;
    public static volatile bool finished;
    static void Thread2() {
        result = 143;
        finished = true;
    }
    static void Main() {
        finished = false;
        // Run Thread2() in a new thread
        new Thread(new ThreadStart(Thread2)).Start();
        // wait for Thread2 to signal that it has a result by setting
        // finished to true.
        for (;;) {
            if (finished) {
                Console.WriteLine("result = {0}", result);
                return;
            }
        }
    }
}
```

produces the output:

```
result = 143
```

In this example, the method `Main` starts a new thread that runs the method `Thread2`. This method stores a value into a non-volatile field called `result`, then stores `true` in the volatile field `finished`. The main thread waits for the field `finished` to be set to `true`, then reads the field `result`. Since `result` has been declared `volatile`, the main thread must read the value 143 from the field `result`. If the field `finished` had not been declared `volatile`, then it would be permissible for the store to `result` to be visible to the main thread *after* the store to `finished`, and hence for the main thread to read the value 0 from the field `result`. Declaring `finished` as a `volatile` field prevents any such inconsistency. *end example*]

17.4.4 Field initialization

The initial value of a field, whether it be a static field or an instance field, is the default value (§12.2) of the field’s type. It is not possible to observe the value of a field before this default initialization has occurred, and a field is thus never “uninitialized”. [*Example*: The example


```

1      using System;
2      class Test
3      {
4          static bool b;
5          int i;
6
7          static void Main() {
8              Test t = new Test();
9              Console.WriteLine("b = {0}, i = {1}", b, t.i);
10         }
11     }

```

11 produces the output

12 b = False, i = 0

13 because b and i are both automatically initialized to default values. *end example*]

14 17.4.5 Variable initializers

15 Field declarations may include *variable-initializers*. For static fields, variable initializers correspond to
 16 assignment statements that are executed during class initialization. For instance fields, variable initializers
 17 correspond to assignment statements that are executed when an instance of the class is created.

18 [*Example:* The example

```

19      using System;
20      class Test
21      {
22          static double x = Math.Sqrt(2.0);
23          int i = 100;
24          string s = "Hello";
25
26          static void Main() {
27              Test a = new Test();
28              Console.WriteLine("x = {0}, i = {1}, s = {2}", x, a.i, a.s);
29          }
30      }

```

30 produces the output

31 x = 1.4142135623731, i = 100, s = Hello

32 because an assignment to x occurs when static field initializers execute and assignments to i and s occur
 33 when the instance field initializers execute. *end example*]

34 The default value initialization described in §17.4.3 occurs for all fields, including fields that have variable
 35 initializers. Thus, when a class is initialized, all static fields in that class are first initialized to their default
 36 values, and then the static field initializers are executed in textual order. Likewise, when an instance of a
 37 class is created, all instance fields in that instance are first initialized to their default values, and then the
 38 instance field initializers are executed in textual order.

39 It is possible for static fields with variable initializers to be observed in their default value state. [*Example:*
 40 However, this is strongly discouraged as a matter of style. The example

```

41      using System;
42      class Test
43      {
44          static int a = b + 1;
45          static int b = a + 1;
46
47          static void Main() {
48              Console.WriteLine("a = {0}, b = {1}", a, b);
49          }
50      }

```

50 exhibits this behavior. Despite the circular definitions of a and b, the program is valid. It results in the output

51 a = 1, b = 2

1 because the static fields `a` and `b` are initialized to 0 (the default value for `int`) before their initializers are
 2 executed. When the initializer for `a` runs, the value of `b` is zero, and so `a` is initialized to 1. When the
 3 initializer for `b` runs, the value of `a` is already 1, and so `b` is initialized to 2. *end example*]

4 17.4.5.1 Static field initialization

5 The static field variable initializers of a class correspond to a sequence of assignments that are executed in
 6 the textual order in which they appear in the class declaration. If a static constructor (§17.11) exists in the
 7 class, execution of the static field initializers occurs immediately prior to executing that static constructor.
 8 Otherwise, the static field initializers are executed at an implementation-dependent time prior to the first use
 9 of a static field of that class. [Example: The example

```
10     using System;
11     class Test
12     {
13         static void Main() {
14             Console.WriteLine("{0} {1}", B.Y, A.X);
15         }
16         public static int f(string s) {
17             Console.WriteLine(s);
18             return 1;
19         }
20     }
21     class A
22     {
23         public static int x = Test.f("Init A");
24     }
25     class B
26     {
27         public static int Y = Test.f("Init B");
28     }
```

29 might produce either the output:

```
30     Init A
31     Init B
32     1 1
```

33 or the output:

```
34     Init B
35     Init A
36     1 1
```

37 because the execution of `X`'s initializer and `Y`'s initializer could occur in either order; they are only
 38 constrained to occur before the references to those fields. However, in the example:

```
39     using System;
40     class Test {
41         static void Main() {
42             Console.WriteLine("{0} {1}", B.Y, A.X);
43         }
44         public static int f(string s) {
45             Console.WriteLine(s);
46             return 1;
47         }
48     }
49     class A
50     {
51         static A() {}
52         public static int x = Test.f("Init A");
53     }
```

```

1      class B
2      {
3          static B() {}
4          public static int Y = Test.f("Init B");
5      }

```

6 the output must be:

```

7      Init B
8      Init A
9      1 1

```

10 because the rules for when static constructors execute provide that B's static constructor (and hence B's static
 11 field initializers) must run before A's static constructor and field initializers. *end example*]

12 17.4.5.2 Instance field initialization

13 The instance field variable initializers of a class correspond to a sequence of assignments that are executed
 14 immediately upon entry to any one of the instance constructors (§17.10.2) of that class. The variable
 15 initializers are executed in the textual order in which they appear in the class declaration. The class instance
 16 creation and initialization process is described further in §17.10.

17 A variable initializer for an instance field cannot reference the instance being created. Thus, it is a compile-
 18 time error to reference `this` in a variable initializer, as it is a compile-time error for a variable initializer to
 19 reference any instance member through a *simple-name*. [*Example*: In the example

```

20      class A
21      {
22          int x = 1;
23          int y = x + 1;    // Error, reference to instance member of this
24      }

```

25 the variable initializer for `y` results in a compile-time error because it references a member of the instance
 26 being created. *end example*]

27 17.5 Methods

28 A **method** is a member that implements a computation or action that can be performed by an object or class.
 29 Methods are declared using *method-declarations*:

```

30      method-declaration:
31          method-header method-body
32
33      method-header:
34          attributesopt method-modifiersopt return-type member-name ( formal-parameter-listopt )
35
36      method-modifiers:
37          method-modifier
38          method-modifiers method-modifier
39
40      method-modifier:
41          new
42          public
43          protected
44          internal
45          private
46          static
47          virtual
48          sealed
49          override
50          abstract
51          extern

```

```

1      return-type:
2          type
3          void
4
5      member-name:
6          identifier
7          interface-type . identifier
8
9      method-body:
10         block
11         ;

```

A *method-declaration* may include a set of *attributes* (§24) and a valid combination of the four access modifiers (§17.2.3), the *new* (§17.2.2), *static* (§17.5.2), *virtual* (§17.5.3), *override* (§17.5.4), *sealed* (§17.5.5), *abstract* (§17.5.6), and *extern* (§17.5.7) modifiers.

A declaration has a valid combination of modifiers if all of the following are true:

- The declaration includes a valid combination of access modifiers (§17.2.3).
- The declaration does not include the same modifier multiple times.
- The declaration includes at most one of the following modifiers: *static*, *virtual*, and *override*.
- The declaration includes at most one of the following modifiers: *new* and *override*.
- If the declaration includes the *abstract* modifier, then the declaration does not include any of the following modifiers: *static*, *virtual*, or *extern*.
- If the declaration includes the *private* modifier, then the declaration does not include any of the following modifiers: *virtual*, *override*, or *abstract*.
- If the declaration includes the *sealed* modifier, then the declaration also includes the *override* modifier.

The *return-type* of a method declaration specifies the type of the value computed and returned by the method. The *return-type* is *void* if the method does not return a value.

The *member-name* specifies the name of the method. Unless the method is an explicit interface member implementation (§20.4.1), the *member-name* is simply an *identifier*. For an explicit interface member implementation, the *member-name* consists of an *interface-type* followed by a “.” and an *identifier*.

The optional *formal-parameter-list* specifies the parameters of the method (§17.5.1).

The *return-type* and each of the types referenced in the *formal-parameter-list* of a method must be at least as accessible as the method itself (§10.5.4).

For *abstract* and *extern* methods, the *method-body* consists simply of a semicolon. For all other methods, the *method-body* consists of a *block*, which specifies the statements to execute when the method is invoked.

The name and the formal parameter list of a method define the signature (§10.6) of the method. Specifically, the signature of a method consists of its name and the number, modifiers, and types of its formal parameters. The return type is not part of a method’s signature, nor are the names of the formal parameters.

The name of a method must differ from the names of all other non-methods declared in the same class. In addition, the signature of a method must differ from the signatures of all other methods declared in the same class.

17.5.1 Method parameters

The parameters of a method, if any, are declared by the method’s *formal-parameter-list*.

```

1  formal-parameter-list:
2      fixed-parameters
3      fixed-parameters , parameter-array
4      parameter-array
5
6  fixed-parameters:
7      fixed-parameter
8      fixed-parameters , fixed-parameter
9
10 fixed-parameter:
11     attributesopt parameter-modifieropt type identifier
12
13 parameter-modifier:
14     ref
15     out
16
17 parameter-array:
18     attributesopt params array-type identifier

```

The formal parameter list consists of one or more comma-separated parameters of which only the last may be a *parameter-array*.

A *fixed-parameter* consists of an optional set of *attributes* (§24), an optional **ref** or **out** modifier, a *type*, and an *identifier*. Each *fixed-parameter* declares a parameter of the given type with the given name.

A *parameter-array* consists of an optional set of *attributes* (§24), a **params** modifier, an *array-type*, and an *identifier*. A parameter array declares a single parameter of the given array type with the given name. The *array-type* of a parameter array must be a single-dimensional array type (§19.1). In a method invocation, a parameter array permits either a single argument of the given array type to be specified, or it permits zero or more arguments of the array element type to be specified. Parameter arrays are described further in §17.5.1.4.

A method declaration creates a separate declaration space for parameters and local variables. Names are introduced into this declaration space by the formal parameter list of the method and by local variable declarations in the *block* of the method. All names in the declaration space of a method must be unique. Thus, it is a compile-time error for a parameter or local variable to have the same name as another parameter or local variable.

A method invocation (§14.5.5.1) creates a copy, specific to that invocation, of the formal parameters and local variables of the method, and the argument list of the invocation assigns values or variable references to the newly created formal parameters. Within the *block* of a method, formal parameters can be referenced by their identifiers in *simple-name* expressions (§14.5.2).

There are four kinds of formal parameters:

- Value parameters, which are declared without any modifiers.
- Reference parameters, which are declared with the **ref** modifier.
- Output parameters, which are declared with the **out** modifier.
- Parameter arrays, which are declared with the **params** modifier.

[*Note:* As described in §10.6, the **ref** and **out** modifiers are part of a method's signature, but the **params** modifier is not. *end note*]

17.5.1.1 Value parameters

A parameter declared with no modifiers is a value parameter. A value parameter corresponds to a local variable that gets its initial value from the corresponding argument supplied in the method invocation.

When a formal parameter is a value parameter, the corresponding argument in a method invocation must be an expression of a type that is implicitly convertible (§13.1) to the formal parameter type.

A method is permitted to assign new values to a value parameter. Such assignments only affect the local storage location represented by the value parameter—they have no effect on the actual argument given in the method invocation.

17.5.1.2 Reference parameters

A parameter declared with a `ref` modifier is a reference parameter. Unlike a value parameter, a reference parameter does not create a new storage location. Instead, a reference parameter represents the same storage location as the variable given as the argument in the method invocation.

When a formal parameter is a reference parameter, the corresponding argument in a method invocation must consist of the keyword `ref` followed by a *variable-reference* (§12.3.3) of the same type as the formal parameter. A variable must be definitely assigned before it can be passed as a reference parameter.

Within a method, a reference parameter is always considered definitely assigned.

[*Example:* The example

```
using System;
class Test
{
    static void Swap(ref int x, ref int y) {
        int temp = x;
        x = y;
        y = temp;
    }

    static void Main() {
        int i = 1, j = 2;
        Swap(ref i, ref j);
        Console.WriteLine("i = {0}, j = {1}", i, j);
    }
}
```

produces the output

```
i = 2, j = 1
```

For the invocation of `Swap` in `Main`, `x` represents `i` and `y` represents `j`. Thus, the invocation has the effect of swapping the values of `i` and `j`. *end example*]

In a method that takes reference parameters, it is possible for multiple names to represent the same storage location. [*Example:* In the example

```
class A
{
    string s;

    void F(ref string a, ref string b) {
        s = "One";
        a = "Two";
        b = "Three";
    }

    void G() {
        F(ref s, ref s);
    }
}
```

the invocation of `F` in `G` passes a reference to `s` for both `a` and `b`. Thus, for that invocation, the names `s`, `a`, and `b` all refer to the same storage location, and the three assignments all modify the instance field `s`. *end example*]

17.5.1.3 Output parameters

A parameter declared with an `out` modifier is an output parameter. Similar to a reference parameter, an output parameter does not create a new storage location. Instead, an output parameter represents the same storage location as the variable given as the argument in the method invocation.

When a formal parameter is an output parameter, the corresponding argument in a method invocation must consist of the keyword **out** followed by a *variable-reference* (§12.3.3) of the same type as the formal parameter. A variable need not be definitely assigned before it can be passed as an output parameter, but following an invocation where a variable was passed as an output parameter, the variable is considered definitely assigned.

Within a method, just like a local variable, an output parameter is initially considered unassigned and must be definitely assigned before its value is used.

Every output parameter of a method must be definitely assigned before the method returns.

Output parameters are typically used in methods that produce multiple return values. [*Example:* For example:

```
using System;
class Test
{
    static void SplitPath(string path, out string dir, out string name) {
        int i = path.Length;
        while (i > 0) {
            char ch = path[i - 1];
            if (ch == '\\\' || ch == '/' || ch == ':') break;
            i--;
        }
        dir = path.Substring(0, i);
        name = path.Substring(i);
    }

    static void Main() {
        string dir, name;
        SplitPath("c:\\windows\\system\\hello.txt", out dir, out name);
        Console.WriteLine(dir);
        Console.WriteLine(name);
    }
}
```

The example produces the output:

```
c:\windows\system\
hello.txt
```

Note that the `dir` and `name` variables can be unassigned before they are passed to `SplitPath`, and that they are considered definitely assigned following the call. *end example*

17.5.1.4 Parameter arrays

A parameter declared with a **params** modifier is a parameter array. If a formal parameter list includes a parameter array, it must be the last parameter in the list and it must be of a single-dimensional array type. [*Example:* For example, the types `string[]` and `string[,]` can be used as the type of a parameter array, but the type `string[,]` can not. *end example*] It is not possible to combine the **params** modifier with the modifiers **ref** and **out**.

A parameter array permits arguments to be specified in one of two ways in a method invocation:

- The argument given for a parameter array can be a single expression of a type that is implicitly convertible (§13.1) to the parameter array type. In this case, the parameter array acts precisely like a value parameter.
- Alternatively, the invocation can specify zero or more arguments for the parameter array, where each argument is an expression of a type that is implicitly convertible (§13.1) to the element type of the parameter array. In this case, the invocation creates an instance of the parameter array type with a length corresponding to the number of arguments, initializes the elements of the array instance with the given argument values, and uses the newly created array instance as the actual argument.

Except for allowing a variable number of arguments in an invocation, a parameter array is precisely equivalent to a value parameter (§17.5.1.1) of the same type.

[*Example:* The example

```

using System;
class Test
{
    static void F(params int[] args) {
        Console.WriteLine("Array contains {0} elements:", args.Length);
        foreach (int i in args)
            Console.WriteLine(" {0}", i);
        Console.WriteLine();
    }

    static void Main() {
        int[] arr = {1, 2, 3};
        F(arr);
        F(10, 20, 30, 40);
        F();
    }
}

```

produces the output

```

Array contains 3 elements: 1 2 3
Array contains 4 elements: 10 20 30 40
Array contains 0 elements:

```

The first invocation of `F` simply passes the array `a` as a value parameter. The second invocation of `F` automatically creates a four-element `int[]` with the given element values and passes that array instance as a value parameter. Likewise, the third invocation of `F` creates a zero-element `int[]` and passes that instance as a value parameter. The second and third invocations are precisely equivalent to writing:

```

F(new int[] {10, 20, 30, 40});
F(new int[] {});

```

end example]

When performing overload resolution, a method with a parameter array may be applicable either in its normal form or in its expanded form (§14.4.2.1). The expanded form of a method is available only if the normal form of the method is not applicable and only if a method with the same signature as the expanded form is not already declared in the same type.

[*Example:* The example

```

using System;
class Test
{
    static void F(params object[] a) {
        Console.WriteLine("F(object[])");
    }

    static void F() {
        Console.WriteLine("F()");
    }

    static void F(object a0, object a1) {
        Console.WriteLine("F(object,object)");
    }

    static void Main() {
        F();
        F(1);
        F(1, 2);
        F(1, 2, 3);
        F(1, 2, 3, 4);
    }
}

```

produces the output


```

1      F();
2      F(object[]);
3      F(object,object);
4      F(object[]);
5      F(object[]);

```

In the example, two of the possible expanded forms of the method with a parameter array are already included in the class as regular methods. These expanded forms are therefore not considered when performing overload resolution, and the first and third method invocations thus select the regular methods. When a class declares a method with a parameter array, it is not uncommon to also include some of the expanded forms as regular methods. By doing so it is possible to avoid the allocation of an array instance that occurs when an expanded form of a method with a parameter array is invoked. *end example*

When the type of a parameter array is `object[]`, a potential ambiguity arises between the normal form of the method and the expanded form for a single `object` parameter. The reason for the ambiguity is that an `object[]` is itself implicitly convertible to type `object`. The ambiguity presents no problem, however, since it can be resolved by inserting a cast if needed.

[*Example:* The example

```

17      using System;
18      class Test
19      {
20          static void F(params object[] args) {
21              foreach (object o in args) {
22                  Console.Write(o.GetType().FullName);
23                  Console.Write(" ");
24              }
25              Console.WriteLine();
26          }
27          static void Main() {
28              object[] a = {1, "Hello", 123.456};
29              object o = a;
30              F(a);
31              F((object)a);
32              F(o);
33              F((object[])o);
34          }
35      }

```

produces the output

```

37      System.Int32 System.String System.Double
38      System.Object[]
39      System.Object[]
40      System.Int32 System.String System.Double

```

In the first and last invocations of `F`, the normal form of `F` is applicable because an implicit conversion exists from the argument type to the parameter type (both are of type `object[]`). Thus, overload resolution selects the normal form of `F`, and the argument is passed as a regular value parameter. In the second and third invocations, the normal form of `F` is not applicable because no implicit conversion exists from the argument type to the parameter type (type `object` cannot be implicitly converted to type `object[]`). However, the expanded form of `F` is applicable, so it is selected by overload resolution. As a result, a one-element `object[]` is created by the invocation, and the single element of the array is initialized with the given argument value (which itself is a reference to an `object[]`). *end example*

17.5.2 Static and instance methods

When a method declaration includes a `static` modifier, that method is said to be a static method. When no `static` modifier is present, the method is said to be an instance method.

A static method does not operate on a specific instance, and it is a compile-time error to refer to `this` in a static method.

An instance method operates on a given instance of a class, and that instance can be accessed as `this` (§14.5.7).

When a method is referenced in a *member-access* (§14.5.4) of the form `E.M`, if `M` is a static method, `E` must denote a type that has a method `M`, and if `M` is an instance method, `E` must denote an instance of a type that has a method `M`.

The differences between static and instance members are discussed further in §17.2.5.

17.5.3 Virtual methods

When an instance method declaration includes a `virtual` modifier, that method is said to be a **virtual method**. When no `virtual` modifier is present, the method is said to be a **non-virtual method**.

The implementation of a non-virtual method is invariant: The implementation is the same whether the method is invoked on an instance of the class in which it is declared or an instance of a derived class. In contrast, the implementation of a virtual method can be superseded by derived classes. The process of superseding the implementation of an inherited virtual method is known as **overriding** that method (§17.5.4).

In a virtual method invocation, the **run-time type** of the instance for which that invocation takes place determines the actual method implementation to invoke. In a non-virtual method invocation, the **compile-time type** of the instance is the determining factor. In precise terms, when a method named `N` is invoked with an argument list `A` on an instance with a compile-time type `C` and a run-time type `R` (where `R` is either `C` or a class derived from `C`), the invocation is processed as follows:

- First, overload resolution is applied to `C`, `N`, and `A`, to select a specific method `M` from the set of methods declared in and inherited by `C`. This is described in §14.5.5.1.
- Then, if `M` is a non-virtual method, `M` is invoked.
- Otherwise, `M` is a virtual method, and the most derived implementation of `M` with respect to `R` is invoked.

For every virtual method declared in or inherited by a class, there exists a **most derived implementation** of the method with respect to that class. The most derived implementation of a virtual method `M` with respect to a class `R` is determined as follows:

- If `R` contains the introducing `virtual` declaration of `M`, then this is the most derived implementation of `M`.
- Otherwise, if `R` contains an `override` of `M`, then this is the most derived implementation of `M`.
- Otherwise, the most derived implementation of `M` is the same as that of the direct base class of `R`.

[*Example:* The following example illustrates the differences between virtual and non-virtual methods:

```
using System;
class A
{
    public void F() { Console.WriteLine("A.F"); }
    public virtual void G() { Console.WriteLine("A.G"); }
}
class B: A
{
    new public void F() { Console.WriteLine("B.F"); }
    public override void G() { Console.WriteLine("B.G"); }
}
```

```

1      class Test
2      {
3          static void Main() {
4              B b = new B();
5              A a = b;
6              a.F();
7              b.F();
8              a.G();
9              b.G();
10         }
11     }

```

12 In the example, A introduces a non-virtual method F and a virtual method G. The class B introduces a *new*
 13 non-virtual method F, thus *hiding* the inherited F, and also *overrides* the inherited method G. The example
 14 produces the output:

```

15     A.F
16     B.F
17     B.G
18     B.G

```

19 Notice that the statement `a.G()` invokes `B.G`, not `A.G`. This is because the run-time type of the instance
 20 (which is B), not the compile-time type of the instance (which is A), determines the actual method
 21 implementation to invoke. *end example*

22 Because methods are allowed to hide inherited methods, it is possible for a class to contain several virtual
 23 methods with the same signature. This does not present an ambiguity problem, since all but the most derived
 24 method are hidden. [*Example:* In the example

```

25     using System;
26     class A
27     {
28         public virtual void F() { Console.WriteLine("A.F"); }
29     }
30     class B: A
31     {
32         public override void F() { Console.WriteLine("B.F"); }
33     }
34     class C: B
35     {
36         new public virtual void F() { Console.WriteLine("C.F"); }
37     }
38     class D: C
39     {
40         public override void F() { Console.WriteLine("D.F"); }
41     }
42     class Test
43     {
44         static void Main() {
45             D d = new D();
46             A a = d;
47             B b = d;
48             C c = d;
49             a.F();
50             b.F();
51             c.F();
52             d.F();
53         }
54     }

```

55 the C and D classes contain two virtual methods with the same signature: The one introduced by A and the
 56 one introduced by C. The method introduced by C hides the method inherited from A. Thus, the override
 57 declaration in D overrides the method introduced by C, and it is not possible for D to override the method
 58 introduced by A. The example produces the output:

```

1      B.F
2      B.F
3      D.F
4      D.F

```

5 Note that it is possible to invoke the hidden virtual method by accessing an instance of `D` through a less
6 derived type in which the method is not hidden. *end example*

7 17.5.4 Override methods

8 When an instance method declaration includes an **override** modifier, the method is said to be an **override**
9 **method**. An override method overrides an inherited virtual method with the same signature. Whereas a
10 virtual method declaration *introduces* a new method, an override method declaration *specializes* an existing
11 inherited virtual method by providing a new implementation of that method.

12 The method overridden by an **override** declaration is known as the **overridden base method**. For an
13 override method `M` declared in a class `C`, the overridden base method is determined by examining each base
14 class of `C`, starting with the direct base class of `C` and continuing with each successive direct base class, until
15 an accessible method with the same signature as `M` is located. For the purposes of locating the overridden
16 base method, a method is considered accessible if it is **public**, if it is **protected**, if it is **protected**
17 **internal**, or if it is **internal** and declared in the same program as `C`.

18 A compile-time error occurs unless all of the following are true for an override declaration:

- 19 • An overridden base method can be located as described above.
- 20 • The overridden base method is a virtual, abstract, or override method. In other words, the overridden
21 base method cannot be static or non-virtual.
- 22 • The overridden base method is not a sealed method.
- 23 • The override declaration and the overridden base method have the same declared accessibility. In other
24 words, an override declaration cannot change the accessibility of the virtual method.

25 An override declaration can access the overridden base method using a *base-access* (§14.5.8). [*Example: In*
26 *the example*

```

27     class A
28     {
29         int x;
30         public virtual void PrintFields() {
31             Console.WriteLine("x = {0}", x);
32         }
33     }
34     class B: A
35     {
36         int y;
37         public override void PrintFields() {
38             base.PrintFields();
39             Console.WriteLine("y = {0}", y);
40         }
41     }

```

42 the `base.PrintFields()` invocation in `B` invokes the `PrintFields` method declared in `A`. A *base-*
43 *access* disables the virtual invocation mechanism and simply treats the base method as a non-virtual method.
44 Had the invocation in `B` been written `((A)this).PrintFields()`, it would recursively invoke the
45 `PrintFields` method declared in `B`, not the one declared in `A`, since `PrintFields` is virtual and the run-
46 time type of `((A)this)` is `B`. *end example*

47 Only by including an **override** modifier can a method override another method. In all other cases, a
48 method with the same signature as an inherited method simply hides the inherited method. [*Example: In the*
49 *example*

```

1      class A
2      {
3          public virtual void F() {}
4      }
5      class B: A
6      {
7          public virtual void F() {}    // warning, hiding inherited F()
8      }

```

the `F` method in `B` does not include an `override` modifier and therefore does not override the `F` method in `A`. Rather, the `F` method in `B` hides the method in `A`, and a warning is reported because the declaration does not include a `new` modifier. *end example*]

[*Example:* In the example

```

13     class A
14     {
15         public virtual void F() {}
16     }
17     class B: A
18     {
19         new private void F() {}        // Hides A.F within B
20     }
21     class C: B
22     {
23         public override void F() {}    // Ok, overrides A.F
24     }

```

the `F` method in `B` hides the virtual `F` method inherited from `A`. Since the new `F` in `B` has private access, its scope only includes the class body of `B` and does not extend to `C`. Therefore, the declaration of `F` in `C` is permitted to override the `F` inherited from `A`. *end example*]

17.5.5 Sealed methods

When an instance method declaration includes a **sealed** modifier, that method is said to be a **sealed method**. A sealed method overrides an inherited virtual method with the same signature. An override method can also be marked with the **sealed** modifier. Use of this modifier prevents a derived class from further overriding the method.

[*Example:* The example

```

34     using System;
35     class A
36     {
37         public virtual void F() {
38             Console.WriteLine("A.F");
39         }
40         public virtual void G() {
41             Console.WriteLine("A.G");
42         }
43     }
44     class B: A
45     {
46         sealed override public void F() {
47             Console.WriteLine("B.F");
48         }
49         override public void G() {
50             Console.WriteLine("B.G");
51         }
52     }

```

```

1      class C: B
2      {
3          override public void G() {
4              Console.WriteLine("C.G");
5          }
6      }

```

7 the class B provides two override methods: an F method that has the **sealed** modifier and a G method that
 8 does not. B's use of the sealed modifier prevents C from further overriding F. *end example*

9 17.5.6 Abstract methods

10 When an instance method declaration includes an **abstract** modifier, that method is said to be an **abstract**
 11 **method**. Although an abstract method is implicitly also a virtual method, it cannot have the modifier
 12 **virtual**.

13 An abstract method declaration introduces a new virtual method but does not provide an implementation of
 14 that method. Instead, non-abstract derived classes are required to provide their own implementation by
 15 overriding that method. Because an abstract method provides no actual implementation, the *method-body* of
 16 an abstract method simply consists of a semicolon.

17 Abstract method declarations are only permitted in abstract classes (§17.1.1.1).

18 [Example: In the example

```

19      public abstract class Shape
20      {
21          public abstract void Paint(Graphics g, Rectangle r);
22      }
23      public class Ellipse: Shape
24      {
25          public override void Paint(Graphics g, Rectangle r) {
26              g.DrawEllipse(r);
27          }
28      }
29      public class Box: Shape
30      {
31          public override void Paint(Graphics g, Rectangle r) {
32              g.DrawRect(r);
33          }
34      }

```

35 the Shape class defines the abstract notion of a geometrical shape object that can paint itself. The Paint
 36 method is abstract because there is no meaningful default implementation. The Ellipse and Box classes
 37 are concrete Shape implementations. Because these classes are non-abstract, they are required to override
 38 the Paint method and provide an actual implementation. *end example*

39 It is a compile-time error for a *base-access* (§14.5.8) to reference an abstract method. [Example: In the
 40 example

```

41      abstract class A
42      {
43          public abstract void F();
44      }
45      class B: A
46      {
47          public override void F() {
48              base.F();           // Error, base.F is abstract
49          }
50      }

```

51 a compile-time error is reported for the `base.F()` invocation because it references an abstract method. *end*
 52 *example*

An abstract method declaration is permitted to override a virtual method. This allows an abstract class to force re-implementation of the method in derived classes, and makes the original implementation of the method unavailable. [Example: In the example

```

using System;
class A
{
    public virtual void F() {
        Console.WriteLine("A.F");
    }
}
abstract class B: A
{
    public abstract override void F();
}
class C: B
{
    public override void F() {
        Console.WriteLine("C.F");
    }
}

```

class A declares a virtual method, class B overrides this method with an abstract method, and class C overrides that abstract method to provide its own implementation. *end example*]

17.5.7 External methods

When a method declaration includes an `extern` modifier, the method is said to be an **external method**. External methods are implemented externally, typically using a language other than C#. Because an external method declaration provides no actual implementation, the *method-body* of an external method simply consists of a semicolon.

The mechanism by which linkage to an external method is achieved, is implementation-defined.

[Example: The following example demonstrates the use of the `extern` modifier in combination with a `DllImport` attribute that specifies the name of the external library in which the method is implemented:

```

using System.Text;
using System.Security.Permissions;
using System.Runtime.InteropServices;
class Path
{
    [DllImport("kernel32", SetLastError=true)]
    static extern bool CreateDirectory(string name, SecurityAttribute sa);
    [DllImport("kernel32", SetLastError=true)]
    static extern bool RemoveDirectory(string name);
    [DllImport("kernel32", SetLastError=true)]
    static extern int GetCurrentDirectory(int bufSize, StringBuilder buf);
    [DllImport("kernel32", SetLastError=true)]
    static extern bool SetCurrentDirectory(string name);
}

```

end example]

17.5.8 Method body

The *method-body* of a method declaration consists of either a *block* or a semicolon.

Abstract and external method declarations do not provide a method implementation, so their method bodies simply consist of a semicolon. For any other method, the method body is a block (§15.2) that contains the statements to execute when that method is invoked.

When the return type of a method is **void**, **return** statements (§15.9.4) in that method's body are not permitted to specify an expression. If execution of the method body of a void method completes normally (that is, control flows off the end of the method body), that method simply returns to its caller.

When the return type of a method is not **void**, each **return** statement in that method body must specify an expression of a type that is implicitly convertible to the return type. The endpoint of the method body of a value-returning method must not be reachable. In other words, in a value-returning method, control is not permitted to flow off the end of the method body.

[*Example:* In the example

```
class A
{
    public int F() {}           // Error, return value required
    public int G() {
        return 1;
    }
    public int H(bool b) {
        if (b) {
            return 1;
        }
        else {
            return 0;
        }
    }
}
```

the value-returning **F** method results in a compile-time error because control can flow off the end of the method body. The **G** and **H** methods are correct because all possible execution paths end in a **return** statement that specifies a return value. *end example*]

17.5.9 Method overloading

The method overload resolution rules are described in §14.4.2.

17.6 Properties

A **property** is a member that provides access to an attribute of an object or a class. Examples of properties include the length of a string, the size of a font, the caption of a window, the name of a customer, and so on. Properties are a natural extension of fields—both are named members with associated types, and the syntax for accessing fields and properties is the same. However, unlike fields, properties do not denote storage locations. Instead, properties have **accessors** that specify the statements to be executed when their values are read or written. Properties thus provide a mechanism for associating actions with the reading and writing of an object's attributes; furthermore, they permit such attributes to be computed.

Properties are declared using *property-declarations*:

```
property-declaration:
    attributesopt property-modifiersopt type member-name { accessor-declarations }

property-modifiers:
    property-modifier
    property-modifiers property-modifier
```


property-modifier:

new
public
protected
internal
private
static
virtual
sealed
override
abstract
extern

member-name:

identifier
interface-type . *identifier*

A *property-declaration* may include a set of *attributes* (§24) and a valid combination of the four access modifiers (§17.2.3), the *new* (§17.2.2), *static* (§17.6.1), *virtual* (§17.5.3, §17.6.3), *override* (§17.5.4, §17.6.3), *sealed* (§17.5.5), *abstract* (§17.5.6, §17.6.3), and *extern* modifiers.

Property declarations are subject to the same rules as method declarations (§17.5) with regard to valid combinations of modifiers.

The *type* of a property declaration specifies the type of the property introduced by the declaration, and the *member-name* specifies the name of the property. Unless the property is an explicit interface member implementation, the *member-name* is simply an *identifier*. For an explicit interface member implementation (§20.4.1), the *member-name* consists of an *interface-type* followed by a “.” and an *identifier*.

The *type* of a property must be at least as accessible as the property itself (§10.5.4).

The *accessor-declarations*, which must be enclosed in “{” and “}” tokens, declare the accessors (§17.6.2) of the property. The accessors specify the executable statements associated with reading and writing the property.

Even though the syntax for accessing a property is the same as that for a field, a property is not classified as a variable. Thus, it is not possible to pass a property as a *ref* or *out* argument.

When a property declaration includes an *extern* modifier, the property is said to be an **external property**. Because an external property declaration provides no actual implementation, each of its *accessor-declarations* consists of a semicolon.

17.6.1 Static and instance properties

When a property declaration includes a *static* modifier, the property is said to be a **static property**. When no *static* modifier is present, the property is said to be an **instance property**.

A static property is not associated with a specific instance, and it is a compile-time error to refer to *this* in the accessors of a static property.

An instance property is associated with a given instance of a class, and that instance can be accessed as *this* (§14.5.7) in the accessors of that property.

When a property is referenced in a *member-access* (§14.5.4) of the form *E.M*, if *M* is a static property, *E* must denote a type that has a property *M*, and if *M* is an instance property, *E* must denote an instance having a property *M*.

The differences between static and instance members are discussed further in §17.2.5.

17.6.2 Accessors

The *accessor-declarations* of a property specify the executable statements associated with reading and writing that property.

```

accessor-declarations:
    get-accessor-declaration set-accessor-declarationopt
    set-accessor-declaration get-accessor-declarationopt

get-accessor-declaration:
    attributesopt get accessor-body

set-accessor-declaration:
    attributesopt set accessor-body

accessor-body:
    block
    ;

```

The accessor declarations consist of a *get-accessor-declaration*, a *set-accessor-declaration*, or both. Each accessor declaration consists of the token **get** or **set** followed by an *accessor-body*. For **abstract** and **extern** properties, the *accessor-body* for each accessor specified is simply a semicolon. For the accessors of any non-abstract, non-extern property, the *accessor-body* is a *block* which specifies the statements to be executed when the corresponding accessor is invoked.

A **get** accessor corresponds to a parameterless method with a return value of the property type. Except as the target of an assignment, when a property is referenced in an expression, the **get** accessor of the property is invoked to compute the value of the property (§14.1.1). The body of a **get** accessor must conform to the rules for value-returning methods described in §17.5.8. In particular, all **return** statements in the body of a **get** accessor must specify an expression that is implicitly convertible to the property type. Furthermore, the endpoint of a **get** accessor must not be reachable.

A **set** accessor corresponds to a method with a single value parameter of the property type and a **void** return type. The implicit parameter of a **set** accessor is always named **value**. When a property is referenced as the target of an assignment (§14.13), or as the operand of ++ or -- (§14.5.9, 14.6.5), the **set** accessor is invoked with an argument (whose value is that of the right-hand side of the assignment or the operand of the ++ or -- operator) that provides the new value (§14.13.1). The body of a **set** accessor must conform to the rules for **void** methods described in §17.5.8. In particular, **return** statements in the **set** accessor body are not permitted to specify an expression. Since a **set** accessor implicitly has a parameter named **value**, it is a compile-time error for a local variable declaration in a **set** accessor to have that name.

Based on the presence or absence of the **get** and **set** accessors, a property is classified as follows:

- A property that includes both a **get** accessor and a **set** accessor is said to be a **read-write** property.
- A property that has only a **get** accessor is said to be a **read-only** property. It is a compile-time error for a read-only property to be the target of an assignment.
- A property that has only a **set** accessor is said to be a **write-only** property. Except as the target of an assignment, it is a compile-time error to reference a write-only property in an expression. [Note: The pre- and postfix ++ and -- operators cannot be applied to write-only properties, since these operators read the old value of their operand before they write the new one. *end note*]

[Example: In the example

```

public class Button: Control
{
    private string caption;

```

```

1      public string Caption {
2          get {
3              return caption;
4          }
5          set {
6              if (caption != value) {
7                  caption = value;
8                  Repaint();
9              }
10         }
11     }
12     public override void Paint(Graphics g, Rectangle r) {
13         // Painting code goes here
14     }
15 }

```

the **Button** control declares a public **Caption** property. The **get** accessor of the **Caption** property returns the string stored in the private **caption** field. The **set** accessor checks if the new value is different from the current value, and if so, it stores the new value and repaints the control. Properties often follow the pattern shown above: The **get** accessor simply returns a value stored in a private field, and the **set** accessor modifies that private field and then performs any additional actions required to fully update the state of the object.

Given the **Button** class above, the following is an example of use of the **Caption** property:

```

23     Button okButton = new Button();
24     okButton.Caption = "OK";           // Invokes set accessor
25     string s = okButton.Caption;      // Invokes get accessor

```

Here, the **set** accessor is invoked by assigning a value to the property, and the **get** accessor is invoked by referencing the property in an expression. *end example*

The **get** and **set** accessors of a property are not distinct members, and it is not possible to declare the accessors of a property separately. [Note: As such, it is not possible for the two accessors of a read-write property to have different accessibility. *end note*] [Example: The example

```

31     class A
32     {
33         private string name;
34         public string Name {           // Error, duplicate member name
35             get { return name; }
36         }
37         public string Name {           // Error, duplicate member name
38             set { name = value; }
39         }
40     }

```

does not declare a single read-write property. Rather, it declares two properties with the same name, one read-only and one write-only. Since two members declared in the same class cannot have the same name, the example causes a compile-time error to occur. *end example*

When a derived class declares a property by the same name as an inherited property, the derived property hides the inherited property with respect to both reading and writing. [Example: In the example

```

46     class A
47     {
48         public int P {
49             set {...}
50         }
51     }
52     class B: A
53     {
54         new public int P {
55             get {...}
56         }
57     }

```

the `P` property in `B` hides the `P` property in `A` with respect to both reading and writing. Thus, in the statements

```
B b = new B();
b.P = 1;    // Error, B.P is read-only
((A)b).P = 1; // Ok, reference to A.P
```

the assignment to `b.P` causes a compile-time error to be reported, since the read-only `P` property in `B` hides the write-only `P` property in `A`. Note, however, that a cast can be used to access the hidden `P` property. *end example*

Unlike public fields, properties provide a separation between an object's internal state and its public interface. [*Example*: Consider the example:

```
class Label
{
    private int x, y;
    private string caption;

    public Label(int x, int y, string caption) {
        this.x = x;
        this.y = y;
        this.caption = caption;
    }

    public int X {
        get { return x; }
    }

    public int Y {
        get { return y; }
    }

    public Point Location {
        get { return new Point(x, y); }
    }

    public string Caption {
        get { return caption; }
    }
}
```

Here, the `Label` class uses two `int` fields, `x` and `y`, to store its location. The location is publicly exposed both as an `X` and a `Y` property and as a `Location` property of type `Point`. If, in a future version of `Label`, it becomes more convenient to store the location as a `Point` internally, the change can be made without affecting the public interface of the class:

```
class Label
{
    private Point location;
    private string caption;

    public Label(int x, int y, string caption) {
        this.location = new Point(x, y);
        this.caption = caption;
    }

    public int X {
        get { return location.x; }
    }

    public int Y {
        get { return location.y; }
    }

    public Point Location {
        get { return location; }
    }

    public string Caption {
        get { return caption; }
    }
}
```

1 Had `x` and `y` instead been `public readonly` fields, it would have been impossible to make such a change
 2 to the `Label` class. *end example*

3 [Note: Exposing state through properties is not necessarily any less efficient than exposing fields directly. In
 4 particular, when a property is non-virtual and contains only a small amount of code, the execution
 5 environment may replace calls to accessors with the actual code of the accessors. This process is known as
 6 **inlining**, and it makes property access as efficient as field access, yet preserves the increased flexibility of
 7 properties. *end note*]

8 [Example: Since invoking a `get` accessor is conceptually equivalent to reading the value of a field, it is
 9 considered bad programming style for `get` accessors to have observable side-effects. In the example

```
10     class Counter
11     {
12         private int next;
13         public int Next {
14             get { return next++; }
15         }
16     }
```

17 the value of the `Next` property depends on the number of times the property has previously been accessed.
 18 Thus, accessing the property produces an observable side effect, and the property should be implemented as
 19 a method instead. *end example*

20 [Note: The “no side-effects” convention for `get` accessors doesn’t mean that `get` accessors should always
 21 be written to simply return values stored in fields. Indeed, `get` accessors often compute the value of a
 22 property by accessing multiple fields or invoking methods. However, a properly designed `get` accessor
 23 performs no actions that cause observable changes in the state of the object. *end note*]

24 Properties can be used to delay initialization of a resource until the moment it is first referenced. [Example:
 25 For example:

```
26     using System.IO;
27     public class Console
28     {
29         private static TextReader reader;
30         private static TextWriter writer;
31         private static TextWriter error;
32         public static TextReader In {
33             get {
34                 if (reader == null) {
35                     reader = new StreamReader(Console.OpenStandardInput());
36                 }
37                 return reader;
38             }
39         }
40         public static TextWriter Out {
41             get {
42                 if (writer == null) {
43                     writer = new StreamWriter(Console.OpenStandardOutput());
44                 }
45                 return writer;
46             }
47         }
48         public static TextWriter Error {
49             get {
50                 if (error == null) {
51                     error = new StreamWriter(Console.OpenStandardError());
52                 }
53                 return error;
54             }
55         }
56     }
```

The `Console` class contains three properties, `In`, `Out`, and `Error`, that represent the standard input, output, and error devices, respectively. By exposing these members as properties, the `Console` class can delay their initialization until they are actually used. For example, upon first referencing the `Out` property, as in

```
Console.Out.WriteLine("hello, world");
```

the underlying `TextWriter` for the output device is created. But if the application makes no reference to the `In` and `Error` properties, then no objects are created for those devices. *end example*

17.6.3 Virtual, sealed, override, and abstract accessors

A `virtual` property declaration specifies that the accessors of the property are virtual. The `virtual` modifier applies to both accessors of a read-write property—it is not possible for only one accessor of a read-write property to be virtual.

An `abstract` property declaration specifies that the accessors of the property are virtual, but does not provide an actual implementation of the accessors. Instead, non-abstract derived classes are required to provide their own implementation for the accessors by overriding the property. Because an accessor for an abstract property declaration provides no actual implementation, its *accessor-body* simply consists of a semicolon.

A property declaration that includes both the `abstract` and `override` modifiers specifies that the property is abstract and overrides a base property. The accessors of such a property are also abstract.

Abstract property declarations are only permitted in abstract classes (§17.1.1.1). The accessors of an inherited virtual property can be overridden in a derived class by including a property declaration that specifies an `override` directive. This is known as an **overriding property declaration**. An overriding property declaration does not declare a new property. Instead, it simply specializes the implementations of the accessors of an existing virtual property.

An overriding property declaration must specify the exact same accessibility modifiers, type, and name as the inherited property. If the inherited property has only a single accessor (i.e., if the inherited property is read-only or write-only), the overriding property must include only that accessor. If the inherited property includes both accessors (i.e., if the inherited property is read-write), the overriding property can include either a single accessor or both accessors.

An overriding property declaration may include the `sealed` modifier. Use of this modifier prevents a derived class from further overriding the property. The accessors of a sealed property are also sealed.

Except for differences in declaration and invocation syntax, virtual, sealed, override, and abstract accessors behave exactly like virtual, sealed, override and abstract methods. Specifically, the rules described in §17.5.3, §17.5.4, §17.5.5, and §17.5.6 apply as if accessors were methods of a corresponding form:

- A `get` accessor corresponds to a parameterless method with a return value of the property type and the same modifiers as the containing property.
- A `set` accessor corresponds to a method with a single value parameter of the property type, a `void` return type, and the same modifiers as the containing property.

[*Example:* In the example

```
abstract class A
{
    int y;

    public virtual int x {
        get { return 0; }
    }

    public virtual int y {
        get { return y; }
        set { y = value; }
    }

    public abstract int z { get; set; }
}
```

1 X is a virtual read-only property, Y is a virtual read-write property, and Z is an abstract read-write property.
 2 Because Z is abstract, the containing class A must also be declared abstract.

3 A class that derives from A is show below:

```

4     class B: A
5     {
6         int z;
7         public override int X {
8             get { return base.X + 1; }
9         }
10        public override int Y {
11            set { base.Y = value < 0? 0: value; }
12        }
13        public override int Z {
14            get { return z; }
15            set { z = value; }
16        }
17    }

```

18 Here, the declarations of X, Y, and Z are overriding property declarations. Each property declaration exactly
 19 matches the accessibility modifiers, type, and name of the corresponding inherited property. The **get**
 20 accessor of X and the **set** accessor of Y use the **base** keyword to access the inherited accessors. The
 21 declaration of Z overrides both abstract accessors—thus, there are no outstanding abstract function members
 22 in B, and B is permitted to be a non-abstract class. *end example*]

23 17.7 Events

24 An **event** is a member that enables an object or class to provide notifications. Clients can attach executable
 25 code for events by supplying **event handlers**.

26 Events are declared using *event-declarations*:

```

27     event-declaration:
28         attributesopt event-modifiersopt event type variable-declarators ;
29         attributesopt event-modifiersopt event type member-name { event-accessor-declarations
30     }

31     event-modifiers:
32         event-modifier
33         event-modifiers event-modifier

34     event-modifier:
35         new
36         public
37         protected
38         internal
39         private
40         static
41         virtual
42         sealed
43         override
44         abstract
45         extern

46     event-accessor-declarations:
47         add-accessor-declaration remove-accessor-declaration
48         remove-accessor-declaration add-accessor-declaration

49     add-accessor-declaration:
50         attributesopt add block

```

1 *remove-accessor-declaration:*

2 *attributes*_{opt} **remove** *block*

3 An *event-declaration* may include a set of *attributes* (§24) and a valid combination of the four access
4 modifiers (§17.2.3), the **new** (§17.2.2), **static** (§17.5.2, §17.7.3), **virtual** (§17.5.3, §17.7.4), **override**
5 (§17.5.4, §17.7.4), **sealed** (§17.5.5), **abstract** (§17.5.6, §17.7.4), and **extern** modifiers.

6 Event declarations are subject to the same rules as method declarations (§17.5) with regard to valid
7 combinations of modifiers.

8 The *type* of an event declaration must be a *delegate-type* (§11.2), and that *delegate-type* must be at least as
9 accessible as the event itself (§10.5.4).

10 An event declaration may include *event-accessor-declarations*. However, if it does not, for non-extern, non-
11 abstract events, the compiler shall supply them automatically (§17.7.1); for extern events, the accessors are
12 provided externally.

13 An event declaration that omits *event-accessor-declarations* defines one or more events—one for each of the
14 *variable-declarators*. The attributes and modifiers apply to all of the members declared by such an *event-*
15 *declaration*.

16 It is a compile-time error for an *event-declaration* to include both the **abstract** modifier and brace-
17 delimited *event-accessor-declarations*.

18 When an event declaration includes an **extern** modifier, the event is said to be an **external event**. Because
19 an external event declaration provides no actual implementation, it is an error for it to include both the
20 **extern** modifier and *event-accessor-declarations*.

21 An event can be used as the left-hand operand of the += and -= operators (§14.13.3). These operators are
22 used, respectively, to attach event handlers to, or to remove event handlers from an event, and the access
23 modifiers of the event control the contexts in which such operations are permitted.

24 Since += and -= are the only operations that are permitted on an event outside the type that declares the
25 event, external code can add and remove handlers for an event, but cannot in any other way obtain or modify
26 the underlying list of event handlers.

27 In an operation of the form *x* += *y* or *x* -= *y*, when *x* is an event and the reference takes place outside the
28 type that contains the declaration of *x*, the result of the operation has type **void** (as opposed to having the
29 type of *x*, with the value of *x* after the assignment). This rule prohibits external code from indirectly
30 examining the underlying delegate of an event.

31 [Example: The following example shows how event handlers are attached to instances of the **Button** class:

```
32       public delegate void EventHandler(object sender, EventArgs e);
33       public class Button: Control
34       {
35           public event EventHandler Click;
36       }
37       public class LoginDialog: Form
38       {
39           Button OkButton;
40           Button CancelButton;
41           public LoginDialog() {
42               OkButton = new Button(...);
43               OkButton.Click += new EventHandler(OkButtonClick);
44               CancelButton = new Button(...);
45               CancelButton.Click += new EventHandler(CancelButtonClick);
46           }
47           void OkButtonClick(object sender, EventArgs e) {
48               // Handle OkButton.Click event
49           }
```



```

1      void CancelButtonClick(object sender, EventArgs e) {
2          // Handle CancelButton.Click event
3      }
4  }

```

5 Here, the `LoginDialog` instance constructor creates two `Button` instances and attaches event handlers to the `Click` events. *end example*

7 17.7.1 Field-like events

8 Within the program text of the class or struct that contains the declaration of an event, certain events can be
9 used like fields. To be used in this way, an event must not be **abstract** or **extern**, and must not explicitly
10 include *event-accessor-declarations*. Such an event can be used in any context that permits a field. The field
11 contains a delegate (§22), which refers to the list of event handlers that have been added to the event. If no
12 event handlers have been added, the field contains `null`.

13 [Example: In the example

```

14      public delegate void EventHandler(object sender, EventArgs e);
15      public class Button: Control
16      {
17          public event EventHandler Click;
18          protected void OnClick(EventArgs e) {
19              if (Click != null) Click(this, e);
20          }
21          public void Reset() {
22              Click = null;
23          }
24      }

```

25 `Click` is used as a field within the `Button` class. As the example demonstrates, the field can be examined,
26 modified, and used in delegate invocation expressions. The `OnClick` method in the `Button` class “raises”
27 the `Click` event. The notion of raising an event is precisely equivalent to invoking the delegate represented
28 by the event—thus, there are no special language constructs for raising events. Note that the delegate
29 invocation is preceded by a check that ensures the delegate is non-null.

30 Outside the declaration of the `Button` class, the `Click` member can only be used on the left-hand side of
31 the `+=` and `-=` operators, as in

```
32      b.Click += new EventHandler(...);
```

33 which appends a delegate to the invocation list of the `Click` event, and

```
34      b.Click -= new EventHandler(...);
```

35 which removes a delegate from the invocation list of the `Click` event. *end example*

36 When compiling a field-like event, the compiler automatically creates storage to hold the delegate, and
37 creates accessors for the event that add or remove event handlers to the delegate field. In order to be thread-
38 safe, the addition or removal operations are done while holding the lock (§15.12) on the containing object
39 for an instance event, or the type object (§14.5.11) for a static event.

40 [Note: Thus, an instance event declaration of the form:

```

41      class X {
42          public event D Ev;
43      }

```

44 could be compiled to something equivalent to:

```

45      class X {
46          private D __Ev; // field to hold the delegate
47          public event D Ev {
48              add {
49                  lock(this) { __Ev = __Ev + value; }
50              }

```

```

1         remove {
2             lock(this) { __Ev = __Ev - value; }
3         }
4     }
5 }

```

Within the class `X`, references to `Ev` are compiled to reference the hidden field `__Ev` instead. The name “`__Ev`” is arbitrary; the hidden field could have any name or no name at all.

Similarly, a static event declaration of the form:

```

9     class X {
10         public static event D Ev;
11     }

```

could be compiled to something equivalent to:

```

13     class X {
14         private static D __Ev; // field to hold the delegate
15         public static event D Ev {
16             add {
17                 lock(typeof(X)) { __Ev = __Ev + value; }
18             }
19             remove {
20                 lock(typeof(X)) { __Ev = __Ev - value; }
21             }
22         }
23     }

```

end note]

17.7.2 Event accessors

[*Note:* Event declarations typically omit *event-accessor-declarations*, as in the `Button` example above. One situation for doing so involves the case in which the storage cost of one field per event is not acceptable. In such cases, a class can include *event-accessor-declarations* and use a private mechanism for storing the list of event handlers. Similarly, in cases where the handling of an event requires access to external resources, event accessors may be used to manage these resources. *end note]*

The *event-accessor-declarations* of an event specify the executable statements associated with adding and removing event handlers.

The accessor declarations consist of an *add-accessor-declaration* and a *remove-accessor-declaration*. Each accessor declaration consists of the token `add` or `remove` followed by a *block*. The *block* associated with an *add-accessor-declaration* specifies the statements to execute when an event handler is added, and the *block* associated with a *remove-accessor-declaration* specifies the statements to execute when an event handler is removed.

Each *add-accessor-declaration* and *remove-accessor-declaration* corresponds to a method with a single value parameter of the event type, and a `void` return type. The implicit parameter of an event accessor is named `value`. When an event is used in an event assignment, the appropriate event accessor is used. Specifically, if the assignment operator is `+=` then the add accessor is used, and if the assignment operator is `-=` then the remove accessor is used. In either case, the right-hand operand of the assignment operator is used as the argument to the event accessor. The block of an *add-accessor-declaration* or a *remove-accessor-declaration* must conform to the rules for `void` methods described in §17.5.8. In particular, `return` statements in such a block are not permitted to specify an expression.

Since an event accessor implicitly has a parameter named `value`, it is a compile-time error for a local variable declared in an event accessor to have that name.

[*Example:* In the example

```

1  class Control: Component
2  {
3      // Unique keys for events
4      static readonly object mouseDownEventKey = new object();
5      static readonly object mouseUpEventKey = new object();
6
7      // Return event handler associated with key
8      protected Delegate GetEventHandler(object key) {...}
9
10     // Add event handler associated with key
11     protected void AddEventHandler(object key, Delegate handler) {...}
12
13     // Remove event handler associated with key
14     protected void RemoveEventHandler(object key, Delegate handler) {...}
15
16     // MouseDown event
17     public event MouseEventHandler MouseDown {
18         add { AddEventHandler(mouseDownEventKey, value); }
19         remove { RemoveEventHandler(mouseDownEventKey, value); }
20     }
21
22     // MouseUp event
23     public event MouseEventHandler MouseUp {
24         add { AddEventHandler(mouseUpEventKey, value); }
25         remove { RemoveEventHandler(mouseUpEventKey, value); }
26     }
27
28     // Invoke the MouseUp event
29     protected void OnMouseUp(MouseEventArgs args) {
30         MouseEventHandler handler;
31         handler = (MouseEventHandler)GetEventHandler(mouseUpEventKey);
32         if (handler != null)
33             handler(this, args);
34     }
35 }

```

the `Control` class implements an internal storage mechanism for events. The `AddEventHandler` method associates a delegate value with a key, the `GetEventHandler` method returns the delegate currently associated with a key, and the `RemoveEventHandler` method removes a delegate as an event handler for the specified event. Presumably, the underlying storage mechanism is designed such that there is no cost for associating a `null` delegate value with a key, and thus unhandled events consume no storage. *end example*

17.7.3 Static and instance events

When an event declaration includes a `static` modifier, the event is said to be a **static event**. When no `static` modifier is present, the event is said to be an **instance event**.

A static event is not associated with a specific instance, and it is a compile-time error to refer to `this` in the accessors of a static event.

An instance event is associated with a given instance of a class, and this instance can be accessed as `this` (§14.5.7) in the accessors of that event.

When an event is referenced in a *member-access* (§14.5.4) of the form `E.M`, if `M` is a static event, `E` must denote a type, and if `M` is an instance event, `E` must denote an instance.

The differences between static and instance members are discussed further in §17.2.5.

17.7.4 Virtual, sealed, override, and abstract accessors

A `virtual` event declaration specifies that the accessors of that event are virtual. The `virtual` modifier applies to both accessors of an event.

An **abstract** event declaration specifies that the accessors of the event are virtual, but does not provide an actual implementation of the accessors. Instead, non-abstract derived classes are required to provide their own implementation for the accessors by overriding the event. Because an accessor for an abstract event declaration provides no actual implementation, its *accessor-body* simply consists of a semicolon.

An event declaration that includes both the **abstract** and **override** modifiers specifies that the event is abstract and overrides a base event. The accessors of such an event are also abstract.

Abstract event declarations are only permitted in abstract classes (§17.1.1.1).

The accessors of an inherited virtual event can be overridden in a derived class by including an event declaration that specifies an **override** modifier. This is known as an **overriding event declaration**. An overriding event declaration does not declare a new event. Instead, it simply specializes the implementations of the accessors of an existing virtual event.

An overriding event declaration must specify the exact same accessibility modifiers, type, and name as the overridden event.

An overriding event declaration may include the **sealed** modifier. Use of this modifier prevents a derived class from further overriding the event. The accessors of a sealed event are also sealed.

It is a compile-time error for an overriding event declaration to include a **new** modifier.

Except for differences in declaration and invocation syntax, virtual, sealed, override, and abstract accessors behave exactly like virtual, sealed, override and abstract methods. Specifically, the rules described in §17.5.3, §17.5.4, §17.5.5, and §17.5.6 apply as if accessors were methods of a corresponding form. Each accessor corresponds to a method with a single value parameter of the event type, a **void** return type, and the same modifiers as the containing event.

17.8 Indexers

An **indexer** is a member that enables an object to be indexed in the same way as an array. Indexers are declared using *indexer-declarations*:

indexer-declaration:

*attributes*_{opt} *indexer-modifiers*_{opt} *indexer-declarator* { *accessor-declarations* }

indexer-modifiers:

indexer-modifier

indexer-modifiers *indexer-modifier*

indexer-modifier:

new

public

protected

internal

private

virtual

sealed

override

abstract

extern

indexer-declarator:

type **this** [*formal-parameter-list*]

type *interface-type* . **this** [*formal-parameter-list*]

An *indexer-declaration* may include a set of *attributes* (§24) and a valid combination of the four access modifiers (§17.2.3), the **new** (§17.2.2), **virtual** (§17.5.3), **override** (§17.5.4), **sealed** (§17.5.5), **abstract** (§17.5.6), and **extern** (§17.5.7) modifiers.

Indexer declarations are subject to the same rules as method declarations (§17.5) with regard to valid combinations of modifiers, with the one exception being that the static modifier is not permitted on an indexer declaration.

The modifiers `virtual`, `override`, and `abstract` are mutually exclusive except in one case. The `abstract` and `override` modifiers may be used together so that an abstract indexer can override a virtual one.

The *type* of an indexer declaration specifies the element type of the indexer introduced by the declaration. Unless the indexer is an explicit interface member implementation, the *type* is followed by the keyword `this`. For an explicit interface member implementation, the *type* is followed by an *interface-type*, a `“.”`, and the keyword `this`. Unlike other members, indexers do not have user-defined names.

The *formal-parameter-list* specifies the parameters of the indexer. The formal parameter list of an indexer corresponds to that of a method (§17.5.1), except that at least one parameter must be specified, and that the `ref` and `out` parameter modifiers are not permitted.

The *type* of an indexer and each of the types referenced in the *formal-parameter-list* must be at least as accessible as the indexer itself (§10.5.4).

The *accessor-declarations* (§17.6.2), which must be enclosed in `“{”` and `“}”` tokens, declare the accessors of the indexer. The accessors specify the executable statements associated with reading and writing indexer elements.

Even though the syntax for accessing an indexer element is the same as that for an array element, an indexer element is not classified as a variable. Thus, it is not possible to pass an indexer element as a `ref` or `out` argument.

The *formal-parameter-list* of an indexer defines the signature (§10.6) of the indexer. Specifically, the signature of an indexer consists of the number and types of its formal parameters. The element type and names of the formal parameters are not part of an indexer’s signature.

The signature of an indexer must differ from the signatures of all other indexers declared in the same class.

Indexers and properties are very similar in concept, but differ in the following ways:

- A property is identified by its name, whereas an indexer is identified by its signature.
- A property is accessed through a *simple-name* (§14.5.2) or a *member-access* (§14.5.4), whereas an indexer element is accessed through an *element-access* (§14.5.6.2).
- A property can be a `static` member, whereas an indexer is always an instance member.
- A `get` accessor of a property corresponds to a method with no parameters, whereas a `get` accessor of an indexer corresponds to a method with the same formal parameter list as the indexer.
- A `set` accessor of a property corresponds to a method with a single parameter named `value`, whereas a `set` accessor of an indexer corresponds to a method with the same formal parameter list as the indexer, plus an additional parameter named `value`.
- It is a compile-time error for an indexer accessor to declare a local variable with the same name as an indexer parameter.
- In an overriding property declaration, the inherited property is accessed using the syntax `base.P`, where `P` is the property name. In an overriding indexer declaration, the inherited indexer is accessed using the syntax `base[E]`, where `E` is a comma-separated list of expressions.

Aside from these differences, all rules defined in §17.6.2 and §17.6.3 apply to indexer accessors as well as to property accessors.

When an indexer declaration includes an `extern` modifier, the indexer is said to be an *external indexer*. Because an external indexer declaration provides no actual implementation, each of its *accessor-declarations* consists of a semicolon.

[*Example:* The example below declares a `BitArray` class that implements an indexer for accessing the individual bits in the bit array.

```

1      using System;
2      class BitArray
3      {
4          int[] bits;
5          int length;
6
7          public BitArray(int length) {
8              if (length < 0) throw new ArgumentException();
9              bits = new int[((length - 1) >> 5) + 1];
10             this.length = length;
11         }
12
13         public int Length {
14             get { return length; }
15         }
16
17         public bool this[int index] {
18             get {
19                 if (index < 0 || index >= length) {
20                     throw new IndexOutOfRangeException();
21                 }
22                 return (bits[index >> 5] & 1 << index) != 0;
23             }
24             set {
25                 if (index < 0 || index >= length) {
26                     throw new IndexOutOfRangeException();
27                 }
28                 if (value) {
29                     bits[index >> 5] |= 1 << index;
30                 }
31                 else {
32                     bits[index >> 5] &= ~(1 << index);
33                 }
34             }
35         }
36     }

```

34 An instance of the `BitArray` class consumes substantially less memory than a corresponding `bool[]`
35 (since each value of the former occupies only one bit instead of the latter's one byte), but it permits the same
36 operations as a `bool[]`.

37 The following `CountPrimes` class uses a `BitArray` and the classical "sieve" algorithm to compute the
38 number of primes between 1 and a given maximum:

```

39      class CountPrimes
40      {
41          static int Count(int max) {
42              BitArray flags = new BitArray(max + 1);
43              int count = 1;
44              for (int i = 2; i <= max; i++) {
45                  if (!flags[i]) {
46                      for (int j = i * 2; j <= max; j += i) flags[j] = true;
47                      count++;
48                  }
49              }
50              return count;
51          }
52
53          static void Main(string[] args) {
54              int max = int.Parse(args[0]);
55              int count = Count(max);
56              Console.WriteLine("Found {0} primes between 1 and {1}", count,
57                               max);
58          }
59      }

```

59 Note that the syntax for accessing elements of the `BitArray` is precisely the same as for a `bool[]`. *end*
60 *example*

[Example: The following example shows a 26×10 grid class that has an indexer with two parameters. The first parameter is required to be an upper- or lowercase letter in the range A–Z, and the second is required to be an integer in the range 0–9.

```

using System;
class Grid
{
    const int NumRows = 26;
    const int NumCols = 10;
    int[,] cells = new int[NumRows, NumCols];

    public int this[char c, int colm]
    {
        get {
            c = Char.ToUpper(c);
            if (c < 'A' || c > 'Z') {
                throw new ArgumentException();
            }
            if (colm < 0 || colm >= NumCols) {
                throw new IndexOutOfRangeException();
            }
            return cells[c - 'A', colm];
        }
        set {
            c = Char.ToUpper(c);
            if (c < 'A' || c > 'Z') {
                throw new ArgumentException();
            }
            if (colm < 0 || colm >= NumCols) {
                throw new IndexOutOfRangeException();
            }
            cells[c - 'A', colm] = value;
        }
    }
}

```

end example]

17.8.1 Indexer overloading

The indexer overload resolution rules are described in §14.4.2.

17.9 Operators

An **operator** is a member that defines the meaning of an expression operator that can be applied to instances of the class. Operators are declared using *operator-declarations*:

```

operator-declaration:
    attributesopt operator-modifiers operator-declarator operator-body

operator-modifiers:
    operator-modifier
    operator-modifiers operator-modifier

operator-modifier:
    public
    static
    extern

operator-declarator:
    unary-operator-declarator
    binary-operator-declarator
    conversion-operator-declarator

```

```

1      unary-operator-declarator:
2          type operator overloadable-unary-operator ( type identifier )
3
4      overloadable-unary-operator: one of
5          + - ! ~ ++ -- true false
6
7      binary-operator-declarator:
8          type operator overloadable-binary-operator ( type identifier , type identifier )
9
10     overloadable-binary-operator: one of
11         + - * / % & | ^ << >> == != > < >= <=
12
13     conversion-operator-declarator:
14         implicit operator type ( type identifier )
15         explicit operator type ( type identifier )
16
17     operator-body:
18         block
19     ;

```

There are three categories of overloadable operators: Unary operators (§17.9.1), binary operators (§17.9.2), and conversion operators (§17.9.3).

When an operator declaration includes an **extern** modifier, the operator is said to be an **external operator**. Because an external operator provides no actual implementation, its *operator-body* consists of a semi-colon. For all other operators, the *operator-body* consists of a *block*, which specifies the statements to execute when the operator is invoked. The *block* of an operator must conform to the rules for value-returning methods described in §17.5.8.

The following rules apply to all operator declarations:

- An operator declaration must include both a **public** and a **static** modifier.
- The parameter(s) of an operator must be value parameters. It is a compile-time error for an operator declaration to specify **ref** or **out** parameters.
- The signature of an operator (§17.9.1, §17.9.2, §17.9.3) must differ from the signatures of all other operators declared in the same class.
- All types referenced in an operator declaration must be at least as accessible as the operator itself (§10.5.4).
- It is an error for the same modifier to appear multiple times in an operator declaration.

Each operator category imposes additional restrictions, as described in the following sections.

Like other members, operators declared in a base class are inherited by derived classes. Because operator declarations always require the class or struct in which the operator is declared to participate in the signature of the operator, it is not possible for an operator declared in a derived class to hide an operator declared in a base class. Thus, the **new** modifier is never required, and therefore never permitted, in an operator declaration.

Additional information on unary and binary operators can be found in §14.2.

Additional information on conversion operators can be found in §13.4.

17.9.1 Unary operators

The following rules apply to unary operator declarations, where **T** denotes the class or struct type that contains the operator declaration:

- 1 • A unary `+`, `-`, `!`, or `~` operator must take a single parameter of type `T` and can return any type.
- 2 • A unary `++` or `--` operator must take a single parameter of type `T` and must return type `T`.
- 3 • A unary `true` or `false` operator must take a single parameter of type `T` and must return type `bool`.

4 The signature of a unary operator consists of the operator token (`+`, `-`, `!`, `~`, `++`, `--`, `true`, or `false`) and the
 5 type of the single formal parameter. The return type is not part of a unary operator's signature, nor is the
 6 name of the formal parameter.

7 The `true` and `false` unary operators require pair-wise declaration. A compile-time error occurs if a class
 8 declares one of these operators without also declaring the other. The `true` and `false` operators are
 9 described further in §14.16.

10 *[Example:* The following example shows an implementation and subsequent usage of `operator++` for an
 11 integer vector class:

```

12     public class IntVector
13     {
14         public int Length { ... } // read-only property
15         public int this[int index] { ... } // read-write indexer
16         public IntVector(int vectorLength) { ... }
17         public static IntVector operator++(IntVector iv) {
18             IntVector temp = new IntVector(iv.Length);
19             for (int i = 0; i < iv.Length; ++i)
20                 temp[i] = iv[i] + 1;
21             return temp;
22         }
23     }
24
25     class Test
26     {
27         static void Main() {
28             IntVector iv1 = new IntVector(4); // vector of 4x0
29             IntVector iv2;
30
31             iv2 = iv1++; // iv2 contains 4x0, iv1 contains 4x1
32             iv2 = ++iv1; // iv2 contains 4x2, iv1 contains 4x2
33         }
34     }

```

33 Note how the operator method returns the value produced by adding 1 to the operand, just like the postfix
 34 increment and decrement operators (§14.5.9), and the prefix increment and decrement operators (§14.6.5).
 35 Unlike in C++, this method need not, and, in fact, must not, modify the value of its operand directly. *end*
 36 *example]*

37 17.9.2 Binary operators

38 A binary operator must take two parameters, at least one of which must have the class or struct type in which
 39 the operator is declared. A binary operator can return any type.

40 The signature of a binary operator consists of the operator token (`+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`, `==`, `!=`, `>`, `<`,
 41 `>=`, or `<=`) and the types of the two formal parameters. The return type and the names of the formal
 42 parameters are not part of a binary operator's signature.

43 Certain binary operators require pair-wise declaration. For every declaration of either operator of a pair,
 44 there must be a matching declaration of the other operator of the pair. Two operator declarations match when
 45 they have the same return type and the same type for each parameter. The following operators require pair-
 46 wise declaration:

- 1 • operator == and operator !=
- 2 • operator > and operator <
- 3 • operator >= and operator <=

4 17.9.3 Conversion operators

5 A conversion operator declaration introduces a *user-defined conversion* (§13.4), which augments the pre-defined implicit and explicit conversions.

7 A conversion operator declaration that includes the `implicit` keyword introduces a user-defined implicit conversion. Implicit conversions can occur in a variety of situations, including function member invocations, cast expressions, and assignments. This is described further in §13.1.

10 A conversion operator declaration that includes the `explicit` keyword introduces a user-defined explicit conversion. Explicit conversions can occur in cast expressions, and are described further in §13.2.

12 A conversion operator converts from a source type, indicated by the parameter type of the conversion operator, to a target type, indicated by the return type of the conversion operator. A class or struct is permitted to declare a conversion from a source type *S* to a target type *T* provided all of the following are true:

- 16 • *S* and *T* are different types.
- 17 • Either *S* or *T* is the class or struct type in which the operator declaration takes place.
- 18 • Neither *S* nor *T* is `object` or an *interface-type*.
- 19 • *T* is not a base class of *S*, and *S* is not a base class of *T*.

20 From the second rule it follows that a conversion operator must convert either to or from the class or struct type in which the operator is declared. [*Example:* For example, it is possible for a class or struct type *C* to define a conversion from *C* to `int` and from `int` to *C*, but not from `int` to `bool`. *end example*]

23 It is not possible to redefine a pre-defined conversion. Thus, conversion operators are not allowed to convert from or to `object` because implicit and explicit conversions already exist between `object` and all other types. Likewise, neither the source nor the target types of a conversion can be a base type of the other, since a conversion would then already exist.

27 User-defined conversions are not allowed to convert from or to *interface-types*. In particular, this restriction ensures that no user-defined transformations occur when converting to an *interface-type*, and that a conversion to an *interface-type* succeeds only if the object being converted actually implements the specified *interface-type*.

31 The signature of a conversion operator consists of the source type and the target type. (Note that this is the only form of member for which the return type participates in the signature.) The `implicit` or `explicit` classification of a conversion operator is not part of the operator's signature. Thus, a class or struct cannot declare both an `implicit` and an `explicit` conversion operator with the same source and target types.

35 [*Note:* In general, user-defined implicit conversions should be designed to never throw exceptions and never lose information. If a user-defined conversion can give rise to exceptions (for example, because the source argument is out of range) or loss of information (such as discarding high-order bits), then that conversion should be defined as an explicit conversion. *end note*]

39 [*Example:* In the example

```
40     using System;
41     public struct Digit
42     {
43         byte value;
```

```

1      public Digit(byte value) {
2          if (value < 0 || value > 9) throw new ArgumentException();
3          this.value = value;
4      }
5      public static implicit operator byte(Digit d) {
6          return d.value;
7      }
8      public static explicit operator Digit(byte b) {
9          return new Digit(b);
10     }
11 }

```

12 the conversion from `Digit` to `byte` is implicit because it never throws exceptions or loses information, but
 13 the conversion from `byte` to `Digit` is explicit since `Digit` can only represent a subset of the possible
 14 values of a `byte`. *end example*]

15 17.10 Instance constructors

16 An **instance constructor** is a member that implements the actions required to initialize an instance of a class.
 17 Instance constructors are declared using *constructor-declarations*:

```

18     constructor-declaration:
19         attributesopt constructor-modifiersopt constructor-declarator constructor-body
20
21     constructor-modifiers:
22         constructor-modifier
23         constructor-modifiers constructor-modifier
24
25     constructor-modifier:
26         public
27         protected
28         internal
29         private
30         extern
31
32     constructor-declarator:
33         identifier ( formal-parameter-listopt ) constructor-initializeropt
34
35     constructor-initializer:
36         : base ( argument-listopt )
37         : this ( argument-listopt )
38
39     constructor-body:
40         block
41         ;

```

37 A *constructor-declaration* may include a set of *attributes* (§24), a valid combination of the four access
 38 modifiers (§17.2.3), and an `extern` (§17.5.7) modifier. A constructor declaration is not permitted to include
 39 the same modifier multiple times.

40 The *identifier* of a *constructor-declarator* must name the class in which the instance constructor is declared.
 41 If any other name is specified, a compile-time error occurs.

42 The optional *formal-parameter-list* of an instance constructor is subject to the same rules as the *formal-*
 43 *parameter-list* of a method (§17.5). The formal parameter list defines the signature (§10.6) of an instance
 44 constructor and governs the process whereby overload resolution (§14.4.2) selects a particular instance
 45 constructor in an invocation.

46 Each of the types referenced in the *formal-parameter-list* of an instance constructor must be at least as
 47 accessible as the constructor itself (§10.5.4).

48 The optional *constructor-initializer* specifies another instance constructor to invoke before executing the
 49 statements given in the *constructor-body* of this instance constructor. This is described further in §17.10.1.

When a constructor declaration includes an **extern** modifier, the constructor is said to be an **external constructor**.

Because an external constructor declaration provides no actual implementation, its *constructor-body* consists of a semicolon. For all other constructors, the *constructor-body* consists of a *block*, which specifies the statements to initialize a new instance of the class. This corresponds exactly to the *block* of an instance method with a **void** return type (§17.5.8).

Instance constructors are not inherited. Thus, a class has no instance constructors other than those actually declared in the class. If a class contains no instance constructor declarations, a default instance constructor is automatically provided (§17.10.4).

Instance constructors are invoked by *object-creation-expressions* (§14.5.10.1) and through *constructor-initializers*.

17.10.1 Constructor initializers

All instance constructors (except those for class **object**) implicitly include an invocation of another instance constructor immediately before the *constructor-body*. The constructor to implicitly invoke is determined by the *constructor-initializer*:

- An instance constructor initializer of the form **base**(*argument-list_{opt}*) causes an instance constructor from the direct base class to be invoked. That constructor is selected using *argument-list* and the overload resolution rules of §14.4.2. The set of candidate instance constructors consists of all accessible instance constructors declared in the direct base class. If this set is empty, or if a single best instance constructor cannot be identified, a compile-time error occurs.
- An instance constructor initializer of the form **this**(*argument-list_{opt}*) causes an instance constructor from the class itself to be invoked. The constructor is selected using *argument-list* and the overload resolution rules of §14.4.2. The set of candidate instance constructors consists of all accessible instance constructors declared in the class itself. If that set is empty, or if a single best instance constructor cannot be identified, a compile-time error occurs. If an instance constructor declaration includes a constructor initializer that invokes the constructor itself, a compile-time error occurs.

If an instance constructor has no constructor initializer, a constructor initializer of the form **base**() is implicitly provided. [Note: Thus, an instance constructor declaration of the form

```
C(...) {...}
```

is exactly equivalent to

```
C(...): base() {...}
```

end note]

The scope of the parameters given by the *formal-parameter-list* of an instance constructor declaration includes the constructor initializer of that declaration. Thus, a constructor initializer is permitted to access the parameters of the constructor. [Example: For example:

```
class A
{
    public A(int x, int y) {}
}

class B: A
{
    public B(int x, int y): base(x + y, x - y) {}
}
```

end example]

An instance constructor initializer cannot access the instance being created. Therefore it is a compile-time error to reference **this** in an argument expression of the constructor initializer, as it is a compile-time error for an argument expression to reference any instance member through a *simple-name*.

17.10.2 Instance variable initializers

When an instance constructor has no constructor initializer, or it has a constructor initializer of the form `base(...)`, that constructor implicitly performs the initializations specified by the *variable-initializers* of the instance fields declared in its class. This corresponds to a sequence of assignments that are executed immediately upon entry to the constructor and before the implicit invocation of the direct base class constructor. The variable initializers are executed in the textual order in which they appear in the class declaration.

17.10.3 Constructor execution

Variable initializers are transformed into assignment statements, and these assignment statements are executed *before* the invocation of the base class instance constructor. This ordering ensures that all instance fields are initialized by their variable initializers before *any* statements that have access to that instance are executed. [Example: For example:

```

using System;
class A
{
    public A() {
        PrintFields();
    }
    public virtual void PrintFields() {}
}
class B: A
{
    int x = 1;
    int y;
    public B() {
        y = -1;
    }
    public override void PrintFields() {
        Console.WriteLine("x = {0}, y = {1}", x, y);
    }
}

```

When `new B()` is used to create an instance of B, the following output is produced:

```
x = 1, y = 0
```

The value of `x` is 1 because the variable initializer is executed before the base class instance constructor is invoked. However, the value of `y` is 0 (the default value of an `int`) because the assignment to `y` is not executed until after the base class constructor returns.

It is useful to think of instance variable initializers and constructor initializers as statements that are automatically inserted before the *constructor-body*. [Example: The example

```

using System;
using System.Collections;
class A
{
    int x = 1, y = -1, count;
    public A() {
        count = 0;
    }
    public A(int n) {
        count = n;
    }
}

```

```

1      class B: A
2      {
3          double sqrt2 = Math.Sqrt(2.0);
4          ArrayList items = new ArrayList(100);
5          int max;
6
7          public B(): this(100) {
8              items.Add("default");
9          }
10         public B(int n): base(n - 1) {
11             max = n;
12         }

```

contains several variable initializers; it also contains constructor initializers of both forms (**base** and **this**). The example corresponds to the code shown below, where each comment indicates an automatically inserted statement (the syntax used for the automatically inserted constructor invocations isn't valid, but merely serves to illustrate the mechanism).

```

17     using System.Collections;
18     class A
19     {
20         int x, y, count;
21         public A() {
22             x = 1;                // variable initializer
23             y = -1;               // variable initializer
24             object();              // Invoke object() constructor
25             count = 0;
26         }
27         public A(int n) {
28             x = 1;                // variable initializer
29             y = -1;               // variable initializer
30             object();              // Invoke object() constructor
31             count = n;
32         }
33     }
34     class B: A
35     {
36         double sqrt2;
37         ArrayList items;
38         int max;
39         public B(): this(100) {
40             B(100);                // Invoke B(int) constructor
41             items.Add("default");
42         }
43         public B(int n): base(n - 1) {
44             sqrt2 = Math.Sqrt(2.0); // variable initializer
45             items = new ArrayList(100); // variable initializer
46             A(n - 1);                // Invoke A(int) constructor
47             max = n;
48         }
49     }

```

end example]

17.10.4 Default constructors

If a class contains no instance constructor declarations, a default instance constructor is automatically provided. That default constructor simply invokes the parameterless constructor of the direct base class. If the direct base class does not have an accessible parameterless instance constructor, a compile-time error occurs. If the class is abstract then the declared accessibility for the default constructor is protected. Otherwise, the declared accessibility for the default constructor is public. [Note: Thus, the default constructor is always of the form

```

1      protected C(): base() {}
2  or
3      public C(): base() {}
4  where C is the name of the class. end note

```

5 *[Example: In the example*

```

6      class Message
7      {
8          object sender;
9          string text;
10     }

```

11 a default constructor is provided because the class contains no instance constructor declarations. Thus, the
12 example is precisely equivalent to

```

13     class Message
14     {
15         object sender;
16         string text;
17         public Message(): base() {}
18     }

```

19 *end example*]

20 17.10.5 Private constructors

21 When a class declares only private instance constructors, it is not possible for other classes to derive from
22 that class or to create instances of that class (an exception being classes nested within that class). *[Example:*
23 Private instance constructors are commonly used in classes that contain only static members. For example:

```

24     public class Trig
25     {
26         private Trig() {}    // Prevent instantiation
27         public const double PI = 3.14159265358979323846;
28         public static double Sin(double x) {...}
29         public static double Cos(double x) {...}
30         public static double Tan(double x) {...}
31     }

```

32 The `Trig` class groups related methods and constants, but is not intended to be instantiated. Therefore, it
33 declares a single empty private instance constructor. *end example*] At least one instance constructor must be
34 declared to suppress the automatic generation of a default constructor.

35 17.10.6 Optional instance constructor parameters

36 *[Note: The `this(...)` form of constructor initializer is commonly used in conjunction with overloading to*
37 *implement optional instance constructor parameters. In the example*

```

38     class Text
39     {
40         public Text(): this(0, 0, null) {}
41         public Text(int x, int y): this(x, y, null) {}
42         public Text(int x, int y, string s) {
43             // Actual constructor implementation
44         }
45     }

```

46 the first two instance constructors merely provide the default values for the missing arguments. Both use a
47 `this(...)` constructor initializer to invoke the third instance constructor, which actually does the work of
48 initializing the new instance. The effect is that of optional constructor parameters:

```

49     Text t1 = new Text();           // Same as Text(0, 0, null)
50     Text t2 = new Text(5, 10);     // Same as Text(5, 10, null)
51     Text t3 = new Text(5, 20, "Hello");

```

1 *end note]*

2 **17.11 Static constructors**

3 A **static constructor** is a member that implements the actions required to initialize a class. Static
4 constructors are declared using *static-constructor-declarations*:

```
5      static-constructor-declaration:
6          attributesopt static-constructor-modifiers identifier ( ) static-constructor-body
7
8      static-constructor-modifiers:
9          externopt static
10         static externopt
11
12     static-constructor-body:
13         block
14     ;
```

13 A *static-constructor-declaration* may include a set of *attributes* (§24) and an **extern** modifier (§17.5.7).

14 The *identifier* of a *static-constructor-declaration* must name the class in which the static constructor is
15 declared. If any other name is specified, a compile-time error occurs.

16 When a static constructor declaration includes an **extern** modifier, the static constructor is said to be an
17 **external static constructor**. Because an external static constructor declaration provides no actual
18 implementation, its *static-constructor-body* consists of a semicolon. For all other static constructor
19 declarations, the *static-constructor-body* consists of a *block*, which specifies the statements to execute in
20 order to initialize the class. This corresponds exactly to the *method-body* of a static method with a **void**
21 return type (§17.5.8).

22 Static constructors are not inherited, and cannot be called directly.

23 The static constructor for a class executes at most once in a given application domain. The execution of a
24 static constructor is triggered by the first of the following events to occur within an application domain:

- 25 • An instance of the class is created.
- 26 • Any of the static members of the class are referenced.

27 If a class contains the **Main** method (§10.1) in which execution begins, the static constructor for that class
28 executes before the **Main** method is called. If a class contains any static fields with initializers, those
29 initializers are executed in textual order immediately prior to executing the static constructor.

30 [Example: The example

```
31     using System;
32     class Test
33     {
34         static void Main() {
35             A.F();
36             B.F();
37         }
38     }
39     class A
40     {
41         static A() {
42             Console.WriteLine("Init A");
43         }
44         public static void F() {
45             Console.WriteLine("A.F");
46         }
47     }
```



```

1      class B
2      {
3          static B() {
4              Console.WriteLine("Init B");
5          }
6          public static void F() {
7              Console.WriteLine("B.F");
8          }
9      }

```

10 must produce the output:

```

11      Init A
12      A.F
13      Init B
14      B.F

```

15 because the execution of A's static constructor is triggered by the call to A.F, and the execution of B's static
 16 constructor is triggered by the call to B.F. *end example*]

17 It is possible to construct circular dependencies that allow static fields with variable initializers to be
 18 observed in their default value state.

19 [*Example:* The example

```

20      using System;
21      class A
22      {
23          public static int X;
24          static A() { X = B.Y + 1;}
25      }
26      class B
27      {
28          public static int Y = A.X + 1;
29          static B() {}
30          static void Main() {
31              Console.WriteLine("X = {0}, Y = {1}", A.X, B.Y);
32          }
33      }

```

34 produces the output

```

35      X = 1, Y = 2

```

36 To execute the Main method, the system first runs the initializer for B.Y, prior to class B's static constructor.
 37 Y's initializer causes A's static constructor to be run because the value of A.X is referenced. The static
 38 constructor of A in turn proceeds to compute the value of X, and in doing so fetches the default value of Y,
 39 which is zero. A.X is thus initialized to 1. The process of running A's static field initializers and static
 40 constructor then completes, returning to the calculation of the initial value of Y, the result of which
 41 becomes 2. *end example*]

42 17.12 Destructors

43 A **destructor** is a member that implements the actions required to destruct an instance of a class. A
 44 destructor is declared using a *destructor-declaration*:

```

45      destructor-declaration:
46          attributesopt externopt ~ identifier ( ) destructor-body
47      destructor-body:
48          block
49          ;

```

50 A *destructor-declaration* may include a set of *attributes* (§24).

51 The *identifier* of a *destructor-declaration* must name the class in which the destructor is declared. If any other
 52 name is specified, a compile-time error occurs.

When a destructor declaration includes an **extern** modifier, the destructor is said to be an **external destructor**. Because an external destructor declaration provides no actual implementation, its *destructor-body* consists of a semicolon. For all other destructors, the *destructor-body* consists of a *block*, which specifies the statements to execute in order to destruct an instance of the class. A *destructor-body* corresponds exactly to the *method-body* of an instance method with a **void** return type (§17.5.8).

Destructors are not inherited. Thus, a class has no destructors other than the one which may be declared in that class.

[*Note:* Since a destructor is required to have no parameters, it cannot be overloaded, so a class can have, at most, one destructor. *end note*]

Destructors are invoked automatically, and cannot be invoked explicitly. An instance becomes eligible for destruction when it is no longer possible for any code to use that instance. Execution of the destructor for the instance may occur at any time after the instance becomes eligible for destruction. When an instance is destructed, the destructors in that instance's inheritance chain are called, in order, from most derived to least derived [*Example:* The output of the example

```
using System;
class A
{
    ~A() {
        Console.WriteLine("A's destructor");
    }
}
class B: A
{
    ~B() {
        Console.WriteLine("B's destructor");
    }
}
class Test
{
    static void Main() {
        B b = new B();
        b = null;
        GC.Collect();
        GC.WaitForPendingFinalizers();
    }
}
```

is

```
B's destructor
A's destructor
```

since destructors in an inheritance chain are called in order, from most derived to least derived. *end example*]

Destructors may be implemented by overriding the virtual method **Finalize** on **System.Object**. In any event, C# programs are not permitted to override this method or call it (or overrides of it) directly.

[*Example:* For instance, the program

```
class A
{
    override protected void Finalize() {} // error
    public void F() {
        this.Finalize(); // error
    }
}
```

contains two errors. *end example*]

The compiler behaves as if this method, and overrides of it, does not exist at all. [*Example:* Thus, this program:

```
1      class A
2      {
3          void Finalize() {}      // permitted
4      }
```

5 is valid and the method shown hides `System.Object`'s `Finalize` method. *end example*]

6 For a discussion of the behavior when an exception is thrown from a destructor, see §23.3.

18. Structs

Structs are similar to classes in that they represent data structures that can contain data members and function members. However, unlike classes, structs are value types and do not require heap allocation. A variable of a struct type directly contains the data of the struct, whereas a variable of a class type contains a reference to the data, the latter known as an object.

[*Note:* Structs are particularly useful for small data structures that have value semantics. Complex numbers, points in a coordinate system, or key-value pairs in a dictionary are all good examples of structs. Key to these data structures is that they have few data members, that they do not require use of inheritance or referential identity, and that they can be conveniently implemented using value semantics where assignment copies the value instead of the reference. *end note*]

As described in §11.1.3, the simple types provided by C#, such as `int`, `double`, and `bool`, are, in fact, all struct types. Just as these predefined types are structs, it is also possible to use structs and operator overloading to implement new “primitive” types in the C# language. Two examples of such types are given at the end of this chapter (§18.4).

18.1 Struct declarations

A *struct-declaration* is a *type-declaration* (§16.5) that declares a new struct:

```
struct-declaration:
    attributesopt struct-modifiersopt struct identifier struct-interfacesopt struct-body ;opt
```

A *struct-declaration* consists of an optional set of *attributes* (§24), followed by an optional set of *struct-modifiers* (§18.1.1), followed by the keyword `struct` and an *identifier* that names the struct, followed by an optional *struct-interfaces* specification (§18.1.2), followed by a *struct-body* (§18.1.3), optionally followed by a semicolon.

18.1.1 Struct modifiers

A *struct-declaration* may optionally include a sequence of struct modifiers:

```
struct-modifiers:
    struct-modifier
    struct-modifiers struct-modifier
```

```
struct-modifier:
    new
    public
    protected
    internal
    private
```

It is a compile-time error for the same modifier to appear multiple times in a struct declaration.

The modifiers of a struct declaration have the same meaning as those of a class declaration (§17.1.1).

18.1.2 Struct interfaces

A struct declaration may include a *struct-interfaces* specification, in which case the struct is said to implement the given interface types.

```
struct-interfaces:
    : interface-type-list
```

Interface implementations are discussed further in §20.4.

18.1.3 Struct body

The *struct-body* of a struct defines the members of the struct.

```
struct-body:
    { struct-member-declarationsopt }
```

18.2 Struct members

The members of a struct consist of the members introduced by its *struct-member-declarations* and the members inherited from the type `System.ValueType`.

```
struct-member-declarations:
    struct-member-declaration
    struct-member-declarations struct-member-declaration

struct-member-declaration:
    constant-declaration
    field-declaration
    method-declaration
    property-declaration
    event-declaration
    indexer-declaration
    operator-declaration
    constructor-declaration
    static-constructor-declaration
    type-declaration
```

Except for the differences noted in §18.3, the descriptions of class members provided in §17.2 through §17.11 apply to struct members as well.

18.3 Class and struct differences

18.3.1 Value semantics

Structs are value types (§11.1) and are said to have value semantics. Classes, on the other hand, are reference types (§11.2) and are said to have reference semantics.

A variable of a struct type directly contains the data of the struct, whereas a variable of a class type contains a reference to the data, the latter known as an object.

With classes, it is possible for two variables to reference the same object, and thus possible for operations on one variable to affect the object referenced by the other variable. With structs, the variables each have their own copy of the data, and it is not possible for operations on one to affect the other. Furthermore, because structs are not reference types, it is not possible for values of a struct type to be `null`.

[*Example:* Given the declaration

```
struct Point
{
    public int x, y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

the code fragment

```

1      Point a = new Point(10, 10);
2      Point b = a;
3      a.x = 100;
4      System.Console.WriteLine(b.x);

```

5 outputs the value 10. The assignment of `a` to `b` creates a copy of the value, and `b` is thus unaffected by the
6 assignment to `a.x`. Had `Point` instead been declared as a class, the output would be 100 because `a` and `b`
7 would reference the same object. *end example*]

8 18.3.2 Inheritance

9 All struct types implicitly inherit from `System.ValueType`, which, in turn, inherits from class `object`. A
10 struct declaration may specify a list of implemented interfaces, but it is not possible for a struct declaration
11 to specify a base class.

12 Struct types are never abstract and are always implicitly sealed. The `abstract` and `sealed` modifiers are
13 therefore not permitted in a struct declaration.

14 Since inheritance isn't supported for structs, the declared accessibility of a struct member cannot be
15 `protected` or `protected internal`.

16 Function members in a struct cannot be `abstract` or `virtual`, and the `override` modifier is allowed
17 only to override methods inherited from the type `System.ValueType`.

18 18.3.3 Assignment

19 Assignment to a variable of a struct type creates a *copy* of the value being assigned. This differs from
20 assignment to a variable of a class type, which copies the reference but not the object identified by the
21 reference.

22 Similar to an assignment, when a struct is passed as a value parameter or returned as the result of a function
23 member, a copy of the struct is created. A struct may be passed by reference to a function member using a
24 `ref` or `out` parameter.

25 When a property or indexer of a struct is the target of an assignment, the instance expression associated with
26 the property or indexer access must be classified as a variable. If the instance expression is classified as a
27 value, a compile-time error occurs. This is described in further detail in §14.13.1.

28 18.3.4 Default values

29 As described in §12.2, several kinds of variables are automatically initialized to their default value when
30 they are created. For variables of class types and other reference types, this default value is `null`. However,
31 since structs are value types that cannot be `null`, the default value of a struct is the value produced by
32 setting all value type fields to their default value and all reference type fields to `null`.

33 [*Example:* Referring to the `Point` struct declared above, the example

```

34      Point[] a = new Point[100];

```

35 initializes each `Point` in the array to the value produced by setting the `x` and `y` fields to zero. *end example*]

36 The default value of a struct corresponds to the value returned by the default constructor of the struct
37 (§11.1.1). Unlike a class, a struct is not permitted to declare a parameterless instance constructor. Instead,
38 every struct implicitly has a parameterless instance constructor, which always returns the value that results
39 from setting all value type fields to their default value and all reference type fields to `null`.

40 [*Note:* Structs should be designed to consider the default initialization state a valid state. In the example

```

41      using System;
42      struct KeyValuePair
43      {
44          string key;
45          string value;

```

```

1      public KeyValuePair(string key, string value) {
2          if (key == null || value == null) throw new ArgumentException();
3          this.key = key;
4          this.value = value;
5      }
6  }

```

the user-defined instance constructor protects against null values only where it is explicitly called. In cases where a `KeyValuePair` variable is subject to default value initialization, the `key` and `value` fields will be null, and the struct must be prepared to handle this state. *end note*

18.3.5 Boxing and unboxing

A value of a class type can be converted to type `object` or to an interface type that is implemented by the class simply by treating the reference as another type at compile-time. Likewise, a value of type `object` or a value of an interface type can be converted back to a class type without changing the reference (but of course a run-time type check is required in this case).

Since structs are not reference types, these operations are implemented differently for struct types. When a value of a struct type is converted to type `object` or to an interface type that is implemented by the struct, a boxing operation takes place. Likewise, when a value of type `object` or a value of an interface type is converted back to a struct type, an unboxing operation takes place. A key difference from the same operations on class types is that boxing and unboxing *copies* the struct value either into or out of the boxed instance. [Note: Thus, following a boxing or unboxing operation, changes made to the unboxed struct are not reflected in the boxed struct. *end note*]

For further details on boxing and unboxing, see §11.3.

18.3.6 Meaning of `this`

Within an instance constructor or instance function member of a class, `this` is classified as a value. Thus, while `this` can be used to refer to the instance for which the function member was invoked, it is not possible to assign to `this` in a function member of a class.

Within an instance constructor of a struct, `this` corresponds to an `out` parameter of the struct type, and within an instance function member of a struct, `this` corresponds to a `ref` parameter of the struct type. In both cases, `this` is classified as a variable, and it is possible to modify the entire struct for which the function member was invoked by assigning to `this` or by passing `this` as a `ref` or `out` parameter.

18.3.7 Field initializers

As described in §18.3.4, the default value of a struct consists of the value that results from setting all value type fields to their default value and all reference type fields to `null`. For this reason, a struct does not permit instance field declarations to include variable initializers. [Example: As such, the following example results in one or more compile-time errors:

```

36      struct Point
37      {
38          public int x = 1;    // Error, initializer not permitted
39          public int y = 1;    // Error, initializer not permitted
40      }

```

end example

This restriction applies only to instance fields. Static fields of a struct are permitted to include variable initializers.

18.3.8 Constructors

Unlike a class, a struct is not permitted to declare a parameterless instance constructor. Instead, every struct implicitly has a parameterless instance constructor, which always returns the value that results from setting all value type fields to their default value and all reference type fields to `null` (§11.1.1). A struct can declare instance constructors having parameters. [Example: For example


```

1      struct Point
2      {
3          int x, y;
4          public Point(int x, int y) {
5              this.x = x;
6              this.y = y;
7          }
8      }

```

9 Given the above declaration, the statements

```

10      Point p1 = new Point();
11      Point p2 = new Point(0, 0);

```

12 both create a `Point` with `x` and `y` initialized to zero. *end example*]

13 A struct instance constructor is not permitted to include a constructor initializer of the form `base(...)`.

14 The `this` variable of a struct instance constructor corresponds to an `out` parameter of the struct type, and similar to an `out` parameter, `this` must be definitely assigned (§12.3) at every location where the constructor returns. [*Example*: Consider the instance constructor implementation below:

```

17      struct Point
18      {
19          int x, y;
20          public int X {
21              set { x = value; }
22          }
23          public int Y {
24              set { y = value; }
25          }
26          public Point(int x, int y) {
27              x = x;          // error, this is not yet definitely assigned
28              Y = y;          // error, this is not yet definitely assigned
29          }
30      }

```

31 No instance member function (including the set accessors for the properties `X` and `Y`) can be called until all fields of the struct being constructed have been definitely assigned. Note, however, that if `Point` were a class instead of a struct, the instance constructor implementation would be permitted.

34 *end example*]

35 18.3.9 Destructors

36 A struct is not permitted to declare a destructor.

37 18.4 Struct examples

38 **This whole clause is informative.**

39 18.4.1 Database integer type

40 The `DBInt` struct below implements an integer type that can represent the complete set of values of the `int` type, plus an additional state that indicates an unknown value. A type with these characteristics is commonly used in databases.

```

43      using System;
44      public struct DBInt
45      {
46          // The Null member represents an unknown DBInt value.
47          public static readonly DBInt Null = new DBInt();

```

```

1      // when the defined field is true, this DBInt represents a known value
2      // which is stored in the value field. when the defined field is
3      false,
4      // this DBInt represents an unknown value, and the value field is 0.
5      int value;
6      bool defined;
7      // Private instance constructor. Creates a DBInt with a known value.
8      DBInt(int value) {
9          this.value = value;
10         this.defined = true;
11     }
12     // The IsNull property is true if this DBInt represents an unknown
13     value.
14     public bool IsNull { get { return !defined; } }
15     // The Value property is the known value of this DBInt, or 0 if this
16     // DBInt represents an unknown value.
17     public int value { get { return value; } }
18     // Implicit conversion from int to DBInt.
19     public static implicit operator DBInt(int x) {
20         return new DBInt(x);
21     }
22     // Explicit conversion from DBInt to int. Throws an exception if the
23     // given DBInt represents an unknown value.
24     public static explicit operator int(DBInt x) {
25         if (!x.defined) throw new InvalidOperationException();
26         return x.value;
27     }
28     public static DBInt operator +(DBInt x) {
29         return x;
30     }
31     public static DBInt operator -(DBInt x) {
32         return x.defined? -x.value: Null;
33     }
34     public static DBInt operator +(DBInt x, DBInt y) {
35         return x.defined && y.defined? x.value + y.value: Null;
36     }
37     public static DBInt operator -(DBInt x, DBInt y) {
38         return x.defined && y.defined? x.value - y.value: Null;
39     }
40     public static DBInt operator *(DBInt x, DBInt y) {
41         return x.defined && y.defined? x.value * y.value: Null;
42     }
43     public static DBInt operator /(DBInt x, DBInt y) {
44         return x.defined && y.defined? x.value / y.value: Null;
45     }
46     public static DBInt operator %(DBInt x, DBInt y) {
47         return x.defined && y.defined? x.value % y.value: Null;
48     }
49     public static DBBool operator ==(DBInt x, DBInt y) {
50         return x.defined && y.defined? x.value == y.value: DBBool.Null;
51     }
52     public static DBBool operator !=(DBInt x, DBInt y) {
53         return x.defined && y.defined? x.value != y.value: DBBool.Null;
54     }
55     public static DBBool operator >(DBInt x, DBInt y) {
56         return x.defined && y.defined? x.value > y.value: DBBool.Null;
57     }

```

```

1      public static DBBool operator <(DBInt x, DBInt y) {
2          return x.defined && y.defined? x.value < y.value: DBBool.Null;
3      }
4      public static DBBool operator >=(DBInt x, DBInt y) {
5          return x.defined && y.defined? x.value >= y.value: DBBool.Null;
6      }
7      public static DBBool operator <=(DBInt x, DBInt y) {
8          return x.defined && y.defined? x.value <= y.value: DBBool.Null;
9      }
10     }

```

11 18.4.2 Database boolean type

12 The DBBool struct below implements a three-valued logical type. The possible values of this type are
 13 DBBool.True, DBBool.False, and DBBool.Null, where the Null member indicates an unknown value.
 14 Such three-valued logical types are commonly used in databases.

```

15     using System;
16     public struct DBBool
17     {
18         // The three possible DBBool values.
19         public static readonly DBBool Null = new DBBool(0);
20         public static readonly DBBool False = new DBBool(-1);
21         public static readonly DBBool True = new DBBool(1);
22         // Private field that stores -1, 0, 1 for False, Null, True.
23         sbyte value;
24         // Private instance constructor. The value parameter must be -1, 0, or
25         1.
26         DBBool(int value) {
27             this.value = (sbyte)value;
28         }
29         // Properties to examine the value of a DBBool. Return true if this
30         // DBBool has the given value, false otherwise.
31         public bool IsNull { get { return value == 0; } }
32         public bool IsFalse { get { return value < 0; } }
33         public bool IsTrue { get { return value > 0; } }
34         // Implicit conversion from bool to DBBool. Maps true to DBBool.True
35         and // false to DBBool.False.
36         public static implicit operator DBBool(bool x) {
37             return x? True: False;
38         }
39         // Explicit conversion from DBBool to bool. Throws an exception if the
40         // given DBBool is Null, otherwise returns true or false.
41         public static explicit operator bool(DBBool x) {
42             if (x.value == 0) throw new InvalidOperationException();
43             return x.value > 0;
44         }
45         // Equality operator. Returns Null if either operand is Null,
46         otherwise // returns True or False.
47         public static DBBool operator ==(DBBool x, DBBool y) {
48             if (x.value == 0 || y.value == 0) return Null;
49             return x.value == y.value? True: False;
50         }
51         // Inequality operator. Returns Null if either operand is Null,
52         otherwise // returns True or False.
53         public static DBBool operator !=(DBBool x, DBBool y) {
54             if (x.value == 0 || y.value == 0) return Null;
55             return x.value != y.value? True: False;
56         }
57     }

```

```

1      public static DBBool operator !=(DBBool x, DBBool y) {
2          if (x.value == 0 || y.value == 0) return Null;
3          return x.value != y.value? True: False;
4      }
5      // Logical negation operator. Returns True if the operand is False,
6      Null // if the operand is Null, or False if the operand is True.
7
8      public static DBBool operator !(DBBool x) {
9          return new DBBool(-x.value);
10     }
11     // Logical AND operator. Returns False if either operand is False,
12     // otherwise Null if either operand is Null, otherwise True.
13
14     public static DBBool operator &(DBBool x, DBBool y) {
15         return new DBBool(x.value < y.value? x.value: y.value);
16     }
17     // Logical OR operator. Returns True if either operand is True,
18     otherwise // Null if either operand is Null, otherwise False.
19
20     public static DBBool operator |(DBBool x, DBBool y) {
21         return new DBBool(x.value > y.value? x.value: y.value);
22     }
23     // Definitely true operator. Returns true if the operand is True,
24     false // otherwise.
25
26     public static bool operator true(DBBool x) {
27         return x.value > 0;
28     }
29     // Definitely false operator. Returns true if the operand is False,
30     false // otherwise.
31
32     public static bool operator false(DBBool x) {
33         return x.value < 0;
34     }
35 }

```

End of informative text.

19. Arrays

An array is a data structure that contains a number of variables which are accessed through computed indices. The variables contained in an array, also called the **elements** of the array, are all of the same type, and this type is called the **element type** of the array.

An array has a rank which determines the number of indices associated with each array element. The rank of an array is also referred to as the dimensions of the array. An array with a rank of one is called a **single-dimensional array**. An array with a rank greater than one is called a **multi-dimensional array**. Specific sized multi-dimensional arrays are often referred to as two-dimensional arrays, three-dimensional arrays, and so on. Each dimension of an array has an associated length which is an integral number greater than or equal to zero. The dimension lengths are not part of the type of the array, but rather are established when an instance of the array type is created at run-time. The length of a dimension determines the valid range of indices for that dimension: For a dimension of length N , indices can range from 0 to $N - 1$ inclusive. The total number of elements in an array is the product of the lengths of each dimension in the array. If one or more of the dimensions of an array have a length of zero, the array is said to be empty.

The element type of an array can be any type, including an array type.

19.1 Array types

An array type is written as a *non-array-type* followed by one or more *rank-specifiers*:

```

array-type:
    non-array-type rank-specifiers

non-array-type:
    type

rank-specifiers:
    rank-specifier
    rank-specifiers rank-specifier

rank-specifier:
    [ dim-separatorsopt ]

dim-separators:
    ,
    dim-separators ,

```

A *non-array-type* is any *type* that is not itself an *array-type*.

The rank of an array type is given by the leftmost *rank-specifier* in the *array-type*: A *rank-specifier* indicates that the array is an array with a rank of one plus the number of “,” tokens in the *rank-specifier*.

The element type of an array type is the type that results from deleting the leftmost *rank-specifier*:

- An array type of the form $T[R]$ is an array with rank R and a non-array element type T .
- An array type of the form $T[R][R_1] \dots [R_N]$ is an array with rank R and an element type $T[R_1] \dots [R_N]$.

In effect, the *rank-specifiers* are read from left to right *before* the final non-array element type. [Example: The type `int[][ , ][ , ]` is a single-dimensional array of three-dimensional arrays of two-dimensional arrays of `int`. end example]

At run-time, a value of an array type can be `null` or a reference to an instance of that array type.

19.1.1 The `System.Array` type

The type `System.Array` is the abstract base type of all array types. An implicit reference conversion (§13.1.4) exists from any array type to `System.Array`, and an explicit reference conversion (§13.2.3) exists from `System.Array` to any array type. Note that `System.Array` is not itself an *array-type*. Rather, it is a *class-type* from which all *array-types* are derived.

At run-time, a value of type `System.Array` can be `null` or a reference to an instance of any array type.

19.2 Array creation

Array instances are created by *array-creation-expressions* (§14.5.10.2) or by field or local variable declarations that include an *array-initializer* (§19.6).

When an array instance is created, the rank and length of each dimension are established and then remain constant for the entire lifetime of the instance. In other words, it is not possible to change the rank of an existing array instance, nor is it possible to resize its dimensions.

An array instance is always of an array type. The `System.Array` type is an abstract type that cannot be instantiated.

Elements of arrays created by *array-creation-expressions* are always initialized to their default value (§12.2).

19.3 Array element access

Array elements are accessed using *element-access* expressions (§14.5.6.1) of the form `A[I1, I2, ..., IN]`, where `A` is an expression of an array type and each `Ix` is an expression of type `int`, `uint`, `long`, `ulong`, or of a type that can be implicitly converted to one or more of these types. The result of an array element access is a variable, namely the array element selected by the indices.

The elements of an array can be enumerated using a `foreach` statement (§15.8.4).

19.4 Array members

Every array type inherits the members declared by the `System.Array` type.

19.5 Array covariance

For any two *reference-types* `A` and `B`, if an implicit reference conversion (§13.1.4) or explicit reference conversion (§13.2.3) exists from `A` to `B`, then the same reference conversion also exists from the array type `A[R]` to the array type `B[R]`, where `R` is any given *rank-specifier* (but the same for both array types). This relationship is known as **array covariance**. Array covariance, in particular, means that a value of an array type `A[R]` may actually be a reference to an instance of an array type `B[R]`, provided an implicit reference conversion exists from `B` to `A`.

Because of array covariance, assignments to elements of reference type arrays include a run-time check which ensures that the value being assigned to the array element is actually of a permitted type (§14.13.1).

[*Example:* For example:

```
class Test
{
    static void Fill(object[] array, int index, int count, object value) {
        for (int i = index; i < index + count; i++) array[i] = value;
    }

    static void Main() {
        string[] strings = new string[100];
        Fill(strings, 0, 100, "Undefined");
        Fill(strings, 0, 10, null);
        Fill(strings, 90, 10, 0);
    }
}
```

The assignment to `array[i]` in the `Fill` method implicitly includes a run-time check, which ensures that the object referenced by `value` is either `null` or an instance of a type that is compatible with the actual element type of array. In `Main`, the first two invocations of `Fill` succeed, but the third invocation causes a `System.ArrayTypeMismatchException` to be thrown upon executing the first assignment to `array[i]`. The exception occurs because a boxed `int` cannot be stored in a `string` array. *end example*

Array covariance specifically does not extend to arrays of *value-types*. For example, no conversion exists that permits an `int[]` to be treated as an `object[]`.

19.6 Array initializers

Array initializers may be specified in field declarations (§17.4), local variable declarations (§15.5.1), and array creation expressions (§14.5.10.2):

```

array-initializer:
    { variable-initializer-listopt }
    { variable-initializer-list , }

variable-initializer-list:
    variable-initializer
    variable-initializer-list , variable-initializer

variable-initializer:
    expression
    array-initializer

```

An array initializer consists of a sequence of variable initializers, enclosed by “{” and “}” tokens and separated by “,” tokens. Each variable initializer is an expression or, in the case of a multi-dimensional array, a nested array initializer.

The context in which an array initializer is used determines the type of the array being initialized. In an array creation expression, the array type immediately precedes the initializer. In a field or variable declaration, the array type is the type of the field or variable being declared. When an array initializer is used in a field or variable declaration, [*Example*: such as:

```
int[] a = {0, 2, 4, 6, 8};
```

end example] it is simply shorthand for an equivalent array creation expression: [*Example*:

```
int[] a = new int[] {0, 2, 4, 6, 8};
```

end example]

For a single-dimensional array, the array initializer must consist of a sequence of expressions that are assignment compatible with the element type of the array. The expressions initialize array elements in increasing order, starting with the element at index zero. The number of expressions in the array initializer determines the length of the array instance being created. [*Example*: For example, the array initializer above creates an `int[]` instance of length 5 and then initializes the instance with the following values:

```
a[0] = 0; a[1] = 2; a[2] = 4; a[3] = 6; a[4] = 8;
```

end example]

For a multi-dimensional array, the array initializer must have as many levels of nesting as there are dimensions in the array. The outermost nesting level corresponds to the leftmost dimension and the innermost nesting level corresponds to the rightmost dimension. The length of each dimension of the array is determined by the number of elements at the corresponding nesting level in the array initializer. For each nested array initializer, the number of elements must be the same as the other array initializers at the same level. [*Example*: The example:

```
int[,] b = {{0, 1}, {2, 3}, {4, 5}, {6, 7}, {8, 9}};
```

creates a two-dimensional array with a length of five for the leftmost dimension and a length of two for the rightmost dimension:

1 `int[,] b = new int[5, 2];`

2 and then initializes the array instance with the following values:

```
3            b[0, 0] = 0; b[0, 1] = 1;
4            b[1, 0] = 2; b[1, 1] = 3;
5            b[2, 0] = 4; b[2, 1] = 5;
6            b[3, 0] = 6; b[3, 1] = 7;
7            b[4, 0] = 8; b[4, 1] = 9;
```

8 *end example]*

9 When an array creation expression includes both explicit dimension lengths and an array initializer, the
10 lengths must be constant expressions and the number of elements at each nesting level must match the
11 corresponding dimension length. [*Example:* Here are some examples:

```
12            int i = 3;
13            int[] x = new int[3] {0, 1, 2};        // OK
14            int[] y = new int[i] {0, 1, 2};        // Error, i not a constant
15            int[] z = new int[3] {0, 1, 2, 3};     // Error, length/initializer mismatch
```

16 Here, the initializer for `y` results in a compile-time error because the dimension length expression is not a
17 constant, and the initializer for `z` results in a compile-time error because the length and the number of
18 elements in the initializer do not agree. *end example]*

20. Interfaces

An interface defines a contract. A class or struct that implements an interface must adhere to its contract. An interface may inherit from multiple base interfaces, and a class or struct may implement multiple interfaces.

Interfaces can contain methods, properties, events, and indexers. The interface itself does not provide implementations for the members that it defines. The interface merely specifies the members that must be supplied by classes or interfaces that implement the interface.

20.1 Interface declarations

An *interface-declaration* is a *type-declaration* (§16.5) that declares a new interface type.

interface-declaration:

```
attributesopt interface-modifiersopt interface identifier interface-baseopt interface-body
;opt
```

An *interface-declaration* consists of an optional set of *attributes* (§24), followed by an optional set of *interface-modifiers* (§20.1.1), followed by the keyword `interface` and an *identifier* that names the interface, optionally followed by an optional *interface-base* specification (§20.1.2), followed by a *interface-body* (§20.1.3), optionally followed by a semicolon.

20.1.1 Interface modifiers

An *interface-declaration* may optionally include a sequence of interface modifiers:

interface-modifiers:

```
interface-modifier
interface-modifiers interface-modifier
```

interface-modifier:

```
new
public
protected
internal
private
```

It is a compile-time error for the same modifier to appear multiple times in an interface declaration.

The `new` modifier is only permitted on nested interfaces. It specifies that the interface hides an inherited member by the same name, as described in §17.2.2.

The `public`, `protected`, `internal`, and `private` modifiers control the accessibility of the interface. Depending on the context in which the interface declaration occurs, only some of these modifiers may be permitted (§10.5.1).

20.1.2 Base interfaces

An interface can inherit from zero or more interfaces, which are called the *explicit base interfaces* of the interface. When an interface has one or more explicit base interfaces, then in the declaration of that interface, the interface identifier is followed by a colon and a comma-separated list of base interface identifiers.

interface-base:

```
: interface-type-list
```

The explicit base interfaces of an interface must be at least as accessible as the interface itself (§10.5.4).
 [Note: For example, it is a compile-time error to specify a `private` or `internal` interface in the *interface-base* of a `public` interface. *end note*]

It is a compile-time error for an interface to directly or indirectly inherit from itself.

The **base interfaces** of an interface are the explicit base interfaces and their base interfaces. In other words, the set of base interfaces is the complete transitive closure of the explicit base interfaces, their explicit base interfaces, and so on. An interface inherits all members of its base interfaces. [Example: In the example

```

interface IControl
{
    void Paint();
}

interface ITextBox: IControl
{
    void SetText(string text);
}

interface IListBox: IControl
{
    void SetItems(string[] items);
}

interface IComboBox: ITextBox, IListBox {}
  
```

the base interfaces of `IComboBox` are `IControl`, `ITextBox`, and `IListBox`. In other words, the `IComboBox` interface above inherits members `SetText` and `SetItems` as well as `Paint`. *end example*]

A class or struct that implements an interface also implicitly implements all of the interface's base interfaces.

20.1.3 Interface body

The *interface-body* of an interface defines the members of the interface.

```

interface-body:
    { interface-member-declarationsopt }
  
```

20.2 Interface members

The members of an interface are the members inherited from the base interfaces and the members declared by the interface itself.

```

interface-member-declarations:
    interface-member-declaration
    interface-member-declarations interface-member-declaration

interface-member-declaration:
    interface-method-declaration
    interface-property-declaration
    interface-event-declaration
    interface-indexer-declaration
  
```

An interface declaration may declare zero or more members. The members of an interface must be methods, properties, events, or indexers. An interface cannot contain constants, fields, operators, instance constructors, destructors, or types, nor can an interface contain static members of any kind.

All interface members implicitly have public access. It is a compile-time error for interface member declarations to include any modifiers. In particular, interface members cannot be declared with the modifiers `abstract`, `public`, `protected`, `internal`, `private`, `virtual`, `override`, or `static`.

[Example: The example

```
public delegate void StringListEvent(IStringList sender);
```

```

1      public interface IListString
2      {
3          void Add(string s);
4          int Count { get; }
5          event StringListEvent Changed;
6          string this[int index] { get; set; }
7      }

```

declares an interface that contains one each of the possible kinds of members: A method, a property, an event, and an indexer. *end example*]

An *interface-declaration* creates a new declaration space (§10.3), and the *interface-member-declarations* immediately contained by the *interface-declaration* introduce new members into this declaration space. The following rules apply to *interface-member-declarations*:

- The name of a method must differ from the names of all properties and events declared in the same interface. In addition, the signature (§10.6) of a method must differ from the signatures of all other methods declared in the same interface.
- The name of a property or event must differ from the names of all other members declared in the same interface.
- The signature of an indexer must differ from the signatures of all other indexers declared in the same interface.

The inherited members of an interface are specifically not part of the declaration space of the interface. Thus, an interface is allowed to declare a member with the same name or signature as an inherited member. When this occurs, the derived interface member is said to *hide* the base interface member. Hiding an inherited member is not considered an error, but it does cause the compiler to issue a warning. To suppress the warning, the declaration of the derived interface member must include a *new* modifier to indicate that the derived member is intended to hide the base member. This topic is discussed further in §10.7.1.2.

If a *new* modifier is included in a declaration that doesn't hide an inherited member, a warning is issued to that effect. This warning is suppressed by removing the *new* modifier.

20.2.1 Interface methods

Interface methods are declared using *interface-method-declarations*:

```

interface-method-declaration:
    attributesopt newopt return-type identifier ( formal-parameter-listopt ) ;

```

The *attributes*, *return-type*, *identifier*, and *formal-parameter-list* of an interface method declaration have the same meaning as those of a method declaration in a class (§17.5). An interface method declaration is not permitted to specify a method body, and the declaration therefore always ends with a semicolon.

20.2.2 Interface properties

Interface properties are declared using *interface-property-declarations*:

```

interface-property-declaration:
    attributesopt newopt type identifier { interface-accessors }

interface-accessors:
    attributesopt get ;
    attributesopt set ;
    attributesopt get ; attributesopt set ;
    attributesopt set ; attributesopt get ;

```

The *attributes*, *type*, and *identifier* of an interface property declaration have the same meaning as those of a property declaration in a class (§17.6).

The accessors of an interface property declaration correspond to the accessors of a class property declaration (§17.6.2), except that the accessor body must always be a semicolon. Thus, the accessors simply indicate whether the property is read-write, read-only, or write-only.

20.2.3 Interface events

Interface events are declared using *interface-event-declarations*:

```
interface-event-declaration:
    attributesopt newopt event type identifier ;
```

The *attributes*, *type*, and *identifier* of an interface event declaration have the same meaning as those of an event declaration in a class (§17.7).

20.2.4 Interface indexers

Interface indexers are declared using *interface-indexer-declarations*:

```
interface-indexer-declaration:
    attributesopt newopt type this [ formal-parameter-list ] { interface-accessors }
```

The *attributes*, *type*, and *formal-parameter-list* of an interface indexer declaration have the same meaning as those of an indexer declaration in a class (§17.8).

The accessors of an interface indexer declaration correspond to the accessors of a class indexer declaration (§17.8), except that the accessor body must always be a semicolon. Thus, the accessors simply indicate whether the indexer is read-write, read-only, or write-only.

20.2.5 Interface member access

Interface members are accessed through member access (§14.5.4) and indexer access (§14.5.6.2) expressions of the form *I*.*M* and *I*[*A*], where *I* is an instance of an interface type, *M* is a method, property, or event of that interface type, and *A* is an indexer argument list.

For interfaces that are strictly single-inheritance (each interface in the inheritance chain has exactly zero or one direct base interface), the effects of the member lookup (§14.3), method invocation (§14.5.5.1), and indexer access (§14.5.6.2) rules are exactly the same as for classes and structs: More derived members hide less derived members with the same name or signature. However, for multiple-inheritance interfaces, ambiguities can occur when two or more unrelated base interfaces declare members with the same name or signature. This section shows several examples of such situations. In all cases, explicit casts can be used to resolve the ambiguities.

[*Example:* In the example

```
interface IList
{
    int Count { get; set; }
}

interface ICounter
{
    void Count(int i);
}

interface IListCounter: IList, ICounter {}

class C
{
    void Test(IListCounter x) {
        x.Count(1);           // Error
        x.Count = 1;          // Error
        ((IList)x).Count = 1;  // Ok, invokes IList.Count.set
        ((ICounter)x).Count(1); // Ok, invokes ICounter.Count
    }
}
```

the first two statements cause compile-time errors because the member lookup (§14.3) of `Count` in `IListCounter` is ambiguous. As illustrated by the example, the ambiguity is resolved by casting `x` to the appropriate base interface type. Such casts have no run-time costs—they merely consist of viewing the instance as a less derived type at compile-time. *end example*

[*Example:* In the example

```

interface IInteger
{
    void Add(int i);
}

interface IDouble
{
    void Add(double d);
}

interface INumber: IInteger, IDouble {}

class C
{
    void Test(INumber n) {
        n.Add(1);           // Error, both Add methods are applicable
        n.Add(1.0);         // Ok, only IDouble.Add is applicable
        ((IInteger)n).Add(1); // Ok, only IInteger.Add is a candidate
        ((IDouble)n).Add(1);  // Ok, only IDouble.Add is a candidate
    }
}

```

the invocation `n.Add(1)` is ambiguous because a method invocation (§14.5.5.1) requires all overloaded candidate methods to be declared in the same type. However, the invocation `n.Add(1.0)` is permitted because only `IDouble.Add` is applicable. When explicit casts are inserted, there is only one candidate method, and thus no ambiguity. *end example*

[*Example:* In the example

```

interface IBase
{
    void F(int i);
}

interface ILeft: IBase
{
    new void F(int i);
}

interface IRight: IBase
{
    void G();
}

interface IDerived: ILeft, IRight {}

class A
{
    void Test(IDerived d) {
        d.F(1);           // Invokes ILeft.F
        ((IBase)d).F(1);   // Invokes IBase.F
        ((ILeft)d).F(1);   // Invokes ILeft.F
        ((IRight)d).F(1);  // Invokes IBase.F
    }
}

```

the `IBase.F` member is hidden by the `ILeft.F` member. The invocation `d.F(1)` thus selects `ILeft.F`, even though `IBase.F` appears to not be hidden in the access path that leads through `IRight`.

The intuitive rule for hiding in multiple-inheritance interfaces is simply this: If a member is hidden in any access path, it is hidden in all access paths. Because the access path from `IDerived` to `ILeft` to `IBase` hides `IBase.F`, the member is also hidden in the access path from `IDerived` to `IRight` to `IBase`. *end example*

20.3 Fully qualified interface member names

An interface member is sometimes referred to by its *fully qualified name*. The fully qualified name of an interface member consists of the name of the interface in which the member is declared, followed by a dot, followed by the name of the member. The fully qualified name of a member references the interface in which the member is declared. [Example: For example, given the declarations

```
interface IControl
{
    void Paint();
}

interface ITextBox: IControl
{
    void SetText(string text);
}
```

the fully qualified name of `Paint` is `IControl.Paint` and the fully qualified name of `SetText` is `ITextBox.SetText`. In the example above, it is not possible to refer to `Paint` as `ITextBox.Paint`. *end example*]

When an interface is part of a namespace, the fully qualified name of an interface member includes the namespace name. [Example: For example

```
namespace System
{
    public interface ICloneable
    {
        object Clone();
    }
}
```

Here, the fully qualified name of the `Clone` method is `System.ICloneable.Clone`. *end example*]

20.4 Interface implementations

Interfaces may be implemented by classes and structs. To indicate that a class or struct implements an interface, the interface identifier is included in the base class list of the class or struct. [Example: For example:

```
interface ICloneable
{
    object Clone();
}

interface IComparable
{
    int CompareTo(object other);
}

class ListEntry: ICloneable, IComparable
{
    public object Clone() {...}
    public int CompareTo(object other) {...}
}
```

end example]

A class or struct that implements an interface also implicitly implements all of the interface's base interfaces. This is true even if the class or struct doesn't explicitly list all base interfaces in the base class list. [Example: For example:

```
interface IControl
{
    void Paint();
}
```

```

1      interface ITextBox: IControl
2      {
3          void SetText(string text);
4      }
5      class TextBox: ITextBox
6      {
7          public void Paint() {...}
8          public void SetText(string text) {...}
9      }

```

10 Here, class `TextBox` implements both `IControl` and `ITextBox`. *end example*

11 20.4.1 Explicit interface member implementations

12 For purposes of implementing interfaces, a class or struct may declare **explicit interface member**
13 **implementations**. An explicit interface member implementation is a method, property, event, or indexer
14 declaration that references a fully qualified interface member name. [*Example*: For example

```

15      interface ICloneable
16      {
17          object Clone();
18      }
19      interface IComparable
20      {
21          int CompareTo(object other);
22      }
23      class ListEntry: ICloneable, IComparable
24      {
25          object ICloneable.Clone() {...}
26          int IComparable.CompareTo(object other) {...}
27      }

```

28 Here, `ICloneable.Clone` and `IComparable.CompareTo` are explicit interface member
29 implementations. *end example*

30 [*Example*: In some cases, the name of an interface member may not be appropriate for the implementing
31 class, in which case the interface member may be implemented using explicit interface member
32 implementation. A class implementing a file abstraction, for example, would likely implement a `Close`
33 member function that has the effect of releasing the file resource, and implement the `Dispose` method of
34 the `IDisposable` interface using explicit interface member implementation:

```

35      interface IDisposable {
36          void Dispose();
37      }
38      class MyFile: IDisposable {
39          void IDisposable.Dispose() {
40              Close();
41          }
42          public void Close() {
43              // Do what's necessary to close the file
44              System.GC.SuppressFinalize(this);
45          }
46      }

```

47 *end example*

48 It is not possible to access an explicit interface member implementation through its fully qualified name in a
49 method invocation, property access, or indexer access. An explicit interface member implementation can
50 only be accessed through an interface instance, and is in that case referenced simply by its member name.

51 It is a compile-time error for an explicit interface member implementation to include access modifiers, and it
52 is a compile-time error to include the modifiers `abstract`, `virtual`, `override`, or `static`.

53 Explicit interface member implementations have different accessibility characteristics than other members.
54 Because explicit interface member implementations are never accessible through their fully qualified name

in a method invocation or a property access, they are in a sense private. However, since they can be accessed through an interface instance, they are in a sense also public.

Explicit interface member implementations serve two primary purposes:

- Because explicit interface member implementations are not accessible through class or struct instances, they allow interface implementations to be excluded from the public interface of a class or struct. This is particularly useful when a class or struct implements an internal interface that is of no interest to a consumer of that class or struct.
- Explicit interface member implementations allow disambiguation of interface members with the same signature. Without explicit interface member implementations it would be impossible for a class or struct to have different implementations of interface members with the same signature and return type, as would it be impossible for a class or struct to have any implementation at all of interface members with the same signature but with different return types.

For an explicit interface member implementation to be valid, the class or struct must name an interface in its base class list that contains a member whose fully qualified name, type, and parameter types exactly match those of the explicit interface member implementation. *[Example: Thus, in the following class*

```
class Shape: ICloneable
{
    object ICloneable.Clone() {...}
    int IComparable.CompareTo(object other) {...} // invalid
}
```

the declaration of `IComparable.CompareTo` results in a compile-time error because `IComparable` is not listed in the base class list of `Shape` and is not a base interface of `ICloneable`. Likewise, in the declarations

```
class Shape: ICloneable
{
    object ICloneable.Clone() {...}
}
class Ellipse: Shape
{
    object ICloneable.Clone() {...} // invalid
}
```

the declaration of `ICloneable.Clone` in `Ellipse` results in a compile-time error because `ICloneable` is not explicitly listed in the base class list of `Ellipse`. *end example]*

The fully qualified name of an interface member must reference the interface in which the member was declared. *[Example: Thus, in the declarations*

```
interface IControl
{
    void Paint();
}
interface ITextBox: IControl
{
    void SetText(string text);
}
class TextBox: ITextBox
{
    void IControl.Paint() {...}
    void ITextBox.SetText(string text) {...}
}
```

the explicit interface member implementation of `Paint` must be written as `IControl.Paint`. *end example]*

20.4.2 Interface mapping

A class or struct must provide implementations of all members of the interfaces that are listed in the base class list of the class or struct. The process of locating implementations of interface members in an implementing class or struct is known as *interface mapping*.

Interface mapping for a class or struct *C* locates an implementation for each member of each interface specified in the base class list of *C*. The implementation of a particular interface member *I.M*, where *I* is the interface in which the member *M* is declared, is determined by examining each class or struct *S*, starting with *C* and repeating for each successive base class of *C*, until a match is located:

- If *S* contains a declaration of an explicit interface member implementation that matches *I* and *M*, then this member is the implementation of *I.M*.
- Otherwise, if *S* contains a declaration of a non-static public member that matches *M*, then this member is the implementation of *I.M*.

A compile-time error occurs if implementations cannot be located for all members of all interfaces specified in the base class list of *C*. Note that the members of an interface include those members that are inherited from base interfaces.

For purposes of interface mapping, a class member *A* matches an interface member *B* when:

- *A* and *B* are methods, and the name, type, and formal parameter lists of *A* and *B* are identical.
- *A* and *B* are properties, the name and type of *A* and *B* are identical, and *A* has the same accessors as *B* (*A* is permitted to have additional accessors if it is not an explicit interface member implementation).
- *A* and *B* are events, and the name and type of *A* and *B* are identical.
- *A* and *B* are indexers, the type and formal parameter lists of *A* and *B* are identical, and *A* has the same accessors as *B* (*A* is permitted to have additional accessors if it is not an explicit interface member implementation).

Notable implications of the interface-mapping algorithm are:

- Explicit interface member implementations take precedence over other members in the same class or struct when determining the class or struct member that implements an interface member.
- Neither non-public nor static members participate in interface mapping.

[*Example:* In the example

```
interface ICloneable
{
    object Clone();
}

class C: ICloneable
{
    object ICloneable.Clone() {...}
    public object Clone() {...}
}
```

the `ICloneable.Clone` member of *C* becomes the implementation of `Clone` in `ICloneable` because explicit interface member implementations take precedence over other members. *end example*]

If a class or struct implements two or more interfaces containing a member with the same name, type, and parameter types, it is possible to map each of those interface members onto a single class or struct member.

[*Example:* For example

```
interface IControl
{
    void Paint();
}
```

```

1      interface IForm
2      {
3          void Paint();
4      }
5      class Page: IControl, IForm
6      {
7          public void Paint() {...}
8      }

```

9 Here, the `Paint` methods of both `IControl` and `IForm` are mapped onto the `Paint` method in `Page`. It is
 10 of course also possible to have separate explicit interface member implementations for the two methods. *end*
 11 *example*]

12 If a class or struct implements an interface that contains hidden members, then some members must
 13 necessarily be implemented through explicit interface member implementations. [*Example: For example*

```

14      interface IBase
15      {
16          int P { get; }
17      }
18      interface IDerived: IBase
19      {
20          new int P();
21      }

```

22 An implementation of this interface would require at least one explicit interface member implementation,
 23 and would take one of the following forms

```

24      class C: IDerived
25      {
26          int IBase.P { get {...} }
27          int IDerived.P() {...}
28      }
29      class C: IDerived
30      {
31          public int P { get {...} }
32          int IDerived.P() {...}
33      }
34      class C: IDerived
35      {
36          int IBase.P { get {...} }
37          public int P() {...}
38      }

```

39 *end example*]

40 When a class implements multiple interfaces that have the same base interface, there can be only one
 41 implementation of the base interface. [*Example: In the example*

```

42      interface IControl
43      {
44          void Paint();
45      }
46      interface ITextBox: IControl
47      {
48          void SetText(string text);
49      }
50      interface IListBox: IControl
51      {
52          void SetItems(string[] items);
53      }
54      class ComboBox: IControl, ITextBox, IListBox
55      {
56          void IControl.Paint() {...}

```

```

1      void ITextBox.SetText(string text) {...}
2      void IListBox.SetItems(string[] items) {...}
3  }

```

it is not possible to have separate implementations for the `IControl` named in the base class `list`, the `IControl` inherited by `ITextBox`, and the `IControl` inherited by `IListBox`. Indeed, there is no notion of a separate identity for these interfaces. Rather, the implementations of `ITextBox` and `IListBox` share the same implementation of `IControl`, and `ComboBox` is simply considered to implement three interfaces, `IControl`, `ITextBox`, and `IListBox`. *end example*

The members of a base class participate in interface mapping. [*Example: In the example*

```

10     interface Interface1
11     {
12         void F();
13     }
14     class Class1
15     {
16         public void F() {}
17         public void G() {}
18     }
19     class Class2: Class1, Interface1
20     {
21         new public void G() {}
22     }

```

the method `F` in `Class1` is used in `Class2`'s implementation of `Interface1`. *end example*

20.4.3 Interface implementation inheritance

A class inherits all interface implementations provided by its base classes.

Without explicitly **re-implementing** an interface, a derived class cannot in any way alter the interface mappings it inherits from its base classes. [*Example: For example, in the declarations*

```

28     interface IControl
29     {
30         void Paint();
31     }
32     class Control: IControl
33     {
34         public void Paint() {...}
35     }
36     class TextBox: Control
37     {
38         new public void Paint() {...}
39     }

```

the `Paint` method in `TextBox` hides the `Paint` method in `Control`, but it does not alter the mapping of `Control.Paint` onto `IControl.Paint`, and calls to `Paint` through class instances and interface instances will have the following effects

```

43     Control c = new Control();
44     TextBox t = new TextBox();
45     IControl ic = c;
46     IControl it = t;
47     c.Paint();           // invokes Control.Paint();
48     t.Paint();           // invokes TextBox.Paint();
49     ic.Paint();          // invokes Control.Paint();
50     it.Paint();          // invokes Control.Paint();

```

end example

However, when an interface method is mapped onto a virtual method in a class, it is possible for derived classes to override the virtual method and alter the implementation of the interface. [*Example:* For example, rewriting the declarations above to

```
interface IControl
{
    void Paint();
}

class Control: IControl
{
    public virtual void Paint() {...}
}

class TextBox: Control
{
    public override void Paint() {...}
}
```

the following effects will now be observed

```
Control c = new Control();
TextBox t = new TextBox();
IControl ic = c;
IControl it = t;
c.Paint();           // invokes Control.Paint();
t.Paint();           // invokes TextBox.Paint();
ic.Paint();          // invokes Control.Paint();
it.Paint();          // invokes TextBox.Paint();
```

end example]

Since explicit interface member implementations cannot be declared virtual, it is not possible to override an explicit interface member implementation. However, it is perfectly valid for an explicit interface member implementation to call another method, and that other method can be declared virtual to allow derived classes to override it. [*Example:* For example

```
interface IControl
{
    void Paint();
}

class Control: IControl
{
    void IControl.Paint() { PaintControl(); }
    protected virtual void PaintControl() {...}
}

class TextBox: Control
{
    protected override void PaintControl() {...}
}
```

Here, classes derived from `Control` can specialize the implementation of `IControl.Paint` by overriding the `PaintControl` method. *end example]*

20.4.4 Interface re-implementation

A class that inherits an interface implementation is permitted to **re-implement** the interface by including it in the base class list.

A re-implementation of an interface follows exactly the same interface mapping rules as an initial implementation of an interface. Thus, the inherited interface mapping has no effect whatsoever on the interface mapping established for the re-implementation of the interface. [*Example:* For example, in the declarations

```

1      interface IControl
2      {
3          void Paint();
4      }
5
6      class Control: IControl
7      {
8          void IControl.Paint() {...}
9      }
10
11     class MyControl: Control, IControl
12     {
13         public void Paint() {}
14     }

```

13 the fact that `Control` maps `IControl.Paint` onto `Control.IControl.Paint` doesn't affect the re-
 14 implementation in `MyControl`, which maps `IControl.Paint` onto `MyControl.Paint`. *end example*

15 Inherited public member declarations and inherited explicit interface member declarations participate in the
 16 interface mapping process for re-implemented interfaces. [*Example: For example*

```

17     interface IMethods
18     {
19         void F();
20         void G();
21         void H();
22         void I();
23     }
24
25     class Base: IMethods
26     {
27         void IMethods.F() {}
28         void IMethods.G() {}
29         public void H() {}
30         public void I() {}
31     }
32
33     class Derived: Base, IMethods
34     {
35         public void F() {}
36         void IMethods.H() {}
37     }

```

36 Here, the implementation of `IMethods` in `Derived` maps the interface methods onto `Derived.F`,
 37 `Base.IMethods.G`, `Derived.IMethods.H`, and `Base.I`. *end example*

38 When a class implements an interface, it implicitly also implements all of that interface's base interfaces.
 39 Likewise, a re-implementation of an interface is also implicitly a re-implementation of all of the interface's
 40 base interfaces. [*Example: For example*

```

41     interface IBase
42     {
43         void F();
44     }
45
46     interface IDerived: IBase
47     {
48         void G();
49     }
50
51     class C: IDerived
52     {
53         void IBase.F() {...}
54         void IDerived.G() {...}
55     }
56
57     class D: C, IDerived
58     {
59         public void F() {...}
60         public void G() {...}
61     }

```

1 Here, the re-implementation of `IDerived` also re-implements `IBase`, mapping `IBase.F` onto `D.F`. *end*
 2 *example*]

3 **20.4.5 Abstract classes and interfaces**

4 Like a non-abstract class, an abstract class must provide implementations of all members of the interfaces
 5 that are listed in the base class list of the class. However, an abstract class is permitted to map interface
 6 methods onto abstract methods. [*Example: For example*

```
7     interface IMethods
8     {
9         void F();
10        void G();
11    }
12    abstract class C: IMethods
13    {
14        public abstract void F();
15        public abstract void G();
16    }
```

17 Here, the implementation of `IMethods` maps `F` and `G` onto abstract methods, which must be overridden in
 18 non-abstract classes that derive from `C`. *end example*]

19 Note that explicit interface member implementations cannot be abstract, but explicit interface member
 20 implementations are of course permitted to call abstract methods. [*Example: For example*

```
21     interface IMethods
22     {
23         void F();
24         void G();
25     }
26     abstract class C: IMethods
27     {
28         void IMethods.F() { FF(); }
29         void IMethods.G() { GG(); }
30         protected abstract void FF();
31         protected abstract void GG();
32     }
```

33 Here, non-abstract classes that derive from `C` would be required to override `FF` and `GG`, thus providing the
 34 actual implementation of `IMethods`. *end example*]

21. Enums

An **enum type** is a distinct type that declares a set of named constants. [Example: The example

```
enum Color
{
    Red,
    Green,
    Blue
}
```

declares an enum type named `Color` with members `Red`, `Green`, and `Blue`. *end example*]

21.1 Enum declarations

An enum declaration declares a new enum type. An enum declaration begins with the keyword `enum`, and defines the name, accessibility, underlying type, and members of the enum.

enum-declaration:

attributes_{opt} enum-modifiers_{opt} enum identifier enum-base_{opt} enum-body ;_{opt}

enum-base:

: integral-type

enum-body:

{ enum-member-declarations_{opt} }

{ enum-member-declarations , }

Each enum type has a corresponding integral type called the **underlying type** of the enum type. This underlying type must be able to represent all the enumerator values defined in the enumeration. An enum declaration may explicitly declare an underlying type of `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long` or `ulong`. [Note: `char` cannot be used as an underlying type. *end note*] An enum declaration that does not explicitly declare an underlying type has an underlying type of `int`.

[Example: The example

```
enum Color: long
{
    Red,
    Green,
    Blue
}
```

declares an enum with an underlying type of `long`. *end example*] [Note: A developer might choose to use an underlying type of `long`, as in the example, to enable the use of values that are in the range of `long` but not in the range of `int`, or to preserve this option for the future. *end note*]

21.2 Enum modifiers

An *enum-declaration* may optionally include a sequence of enum modifiers:

enum-modifiers:

enum-modifier

enum-modifiers enum-modifier

```

1      enum-modifier:
2          new
3          public
4          protected
5          internal
6          private

```

It is a compile-time error for the same modifier to appear multiple times in an enum declaration.

The modifiers of an enum declaration have the same meaning as those of a class declaration (§17.1.1). Note, however, that the **abstract** and **sealed** modifiers are not permitted in an enum declaration. Enums cannot be abstract and do not permit derivation.

21.3 Enum members

The body of an enum type declaration defines zero or more enum members, which are the named constants of the enum type. No two enum members can have the same name.

```

14      enum-member-declarations:
15          enum-member-declaration
16          enum-member-declarations , enum-member-declaration
17
18      enum-member-declaration:
19          attributesopt identifier
20          attributesopt identifier = constant-expression

```

Each enum member has an associated constant value. The type of this value is the underlying type for the containing enum. The constant value for each enum member must be in the range of the underlying type for the enum. [Example: The example

```

23      enum Color: uint
24      {
25          Red = -1,
26          Green = -2,
27          Blue = -3
28      }

```

results in a compile-time error because the constant values -1, -2, and -3 are not in the range of the underlying integral type **uint**. *end example*]

Multiple enum members may share the same associated value. [Example: The example

```

32      enum Color
33      {
34          Red,
35          Green,
36          Blue,
37
38          Max = Blue
39      }

```

shows an enum that has two enum members—**Blue** and **Max**—that have the same associated value. *end example*]

The associated value of an enum member is assigned either implicitly or explicitly. If the declaration of the enum member has a *constant-expression* initializer, the value of that constant expression, implicitly converted to the underlying type of the enum, is the associated value of the enum member. If the declaration of the enum member has no initializer, its associated value is set implicitly, as follows:

- If the enum member is the first enum member declared in the enum type, its associated value is zero.
- Otherwise, the associated value of the enum member is obtained by increasing the associated value of the textually preceding enum member by one. This increased value must be within the range of values that can be represented by the underlying type.

[Example: The example

```

using System;

enum Color
{
    Red,
    Green = 10,
    Blue
}

class Test
{
    static void Main() {
        Console.WriteLine(StringFromColor(Color.Red));
        Console.WriteLine(StringFromColor(Color.Green));
        Console.WriteLine(StringFromColor(Color.Blue));
    }

    static string StringFromColor(Color c) {
        switch (c) {
            case Color.Red:
                return String.Format("Red = {0}", (int) c);
            case Color.Green:
                return String.Format("Green = {0}", (int) c);
            case Color.Blue:
                return String.Format("Blue = {0}", (int) c);
            default:
                return "Invalid color";
        }
    }
}

```

prints out the enum member names and their associated values. The output is:

```

Red = 0
Green = 10
Blue = 11

```

for the following reasons:

- the enum member **Red** is automatically assigned the value zero (since it has no initializer and is the first enum member);
- the enum member **Green** is explicitly given the value 10;
- and the enum member **Blue** is automatically assigned the value one greater than the member that textually precedes it.

end example]

The associated value of an enum member may not, directly or indirectly, use the value of its own associated enum member. Other than this circularity restriction, enum member initializers may freely refer to other enum member initializers, regardless of their textual position. Within an enum member initializer, values of other enum members are always treated as having the type of their underlying type, so that casts are not necessary when referring to other enum members.

[Example: The example

```

1      enum Circular
2      {
3          A = B,
4          B
5      }

```

6 results in a compile-time error because the declarations of A and B are circular. A depends on B explicitly,
 7 and B depends on A implicitly. *end example*]

8 Enum members are named and scoped in a manner exactly analogous to fields within classes. The scope of
 9 an enum member is the body of its containing enum type. Within that scope, enum members can be referred
 10 to by their simple name. From all other code, the name of an enum member must be qualified with the name
 11 of its enum type. Enum members do not have any declared accessibility—an enum member is accessible if
 12 its containing enum type is accessible.

13 21.4 Enum values and operations

14 Each enum type defines a distinct type; an explicit enumeration conversion (§13.2.2) is required to convert
 15 between an enum type and an integral type, or between two enum types. The set of values that an enum type
 16 can take on is not limited by its enum members. In particular, any value of the underlying type of an enum
 17 can be cast to the enum type, and is a distinct valid value of that enum type.

18 Enum members have the type of their containing enum type (except within other enum member initializers:
 19 see §21.3). The value of an enum member declared in enum type E with associated value v is (E)v.

20 The following operators can be used on values of enum types: ==, !=, <, >, <=, >= (§14.9.5), + (§14.7.4),
 21 - (§14.7.5), ^, &, | (§14.10.2), ~ (§14.6.4), ++, -- (§14.5.9 and §14.6.5), and sizeof (§25.5.4).

22 Every enum type automatically derives from the class `System.Enum`. Thus, inherited methods and
 23 properties of this class can be used on values of an enum type.